Vision-Language-Action Models

a.k.a. RFM(Robot Foundation Model), LBM(Large Behavior Model)

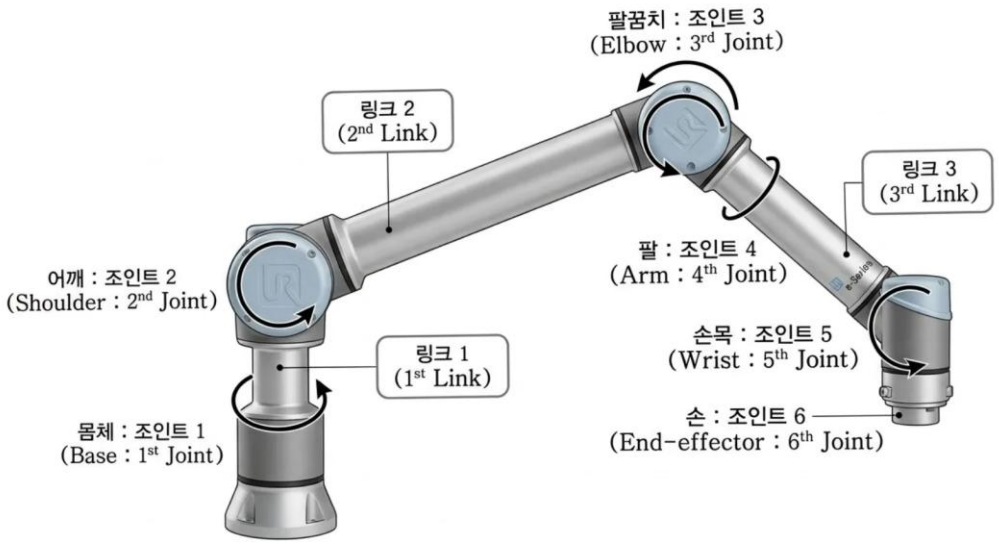
VLA

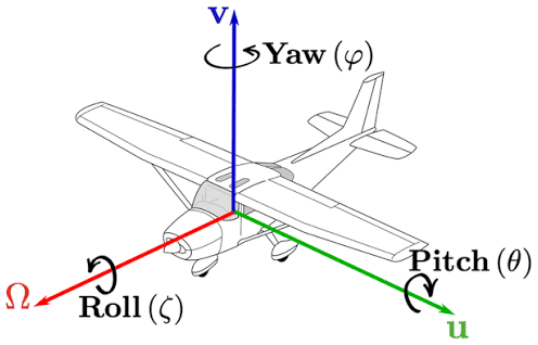
- large-scale AI models
  - trained on massive, multimodal datasets (internet data, video, robot actions)
  - to enable ***versatile, generalizable, and autonomous*** robotic behaviors across various tasks,
  - shifting from task-specific programming to adaptive "Physical AI".
- They interpret sensor data and *natural language* to generate actions.
- Key Aspects
  - Vision-Language-Action (VLA) Models: These models (e.g., RT-2, Pi0) map visual observations and language commands directly to ***robotic actions***.
  - *Versatility & Generalization*: Designed to handle diverse tasks (picking, packing, folding) ***without specific retraining*** for each new task.
  - Data Sources: Trained on diverse data, including the Open X-Embodiment dataset and internet-scale vision/text data.
  - Architecture: Often use Transformer architectures for ***high-level reasoning*** and diffusion or flow matching for continuous, dexterous ***motor control***.
  - Emergent Abilities: Capable of ***zero-shot transfer***, allowing robots to perform tasks they were not explicitly trained on.

VLA Output Format

Action Representation

Robot Action, DoF









➢ DoF: 독립적으로 제어 할 수 있는 움직임의 수

- 각 관절의 각도
- 각 관절의 변화량
- 각 관절의 이동 속도
- Etc.

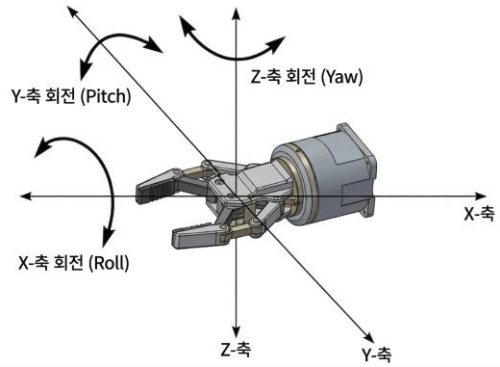
End Effector, FK/IK

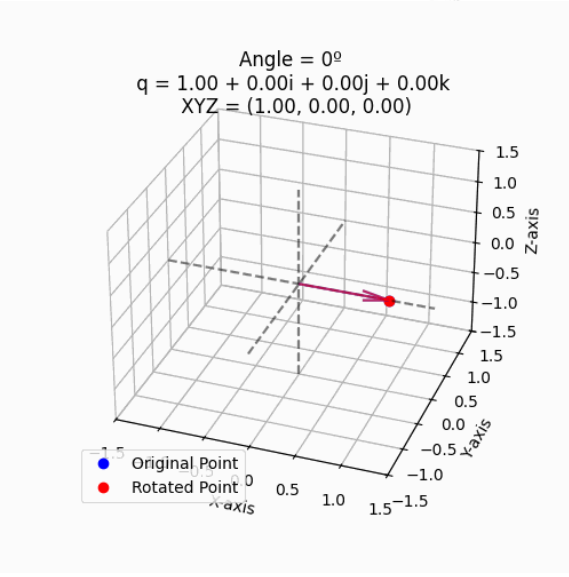
- Robot 움직임에 대한 2가지 생각
  - Ex) “빨간색 컵을 집어서 싱크대에 넣으세요”

    (1) 로봇의 실제 움직임 == **joint** 값이 변한다

    (2) 로봇 움직임에 대한 연상 → 손끝 (end effector: 로봇이 환경과 실제 상호작용하는 끝부분)이 이동해야 하는 **위치와 자세**를 먼저 연상함
    - 위치: 손 끝의 공간상 위치 (x, y, z)
    - 자세: 손 끝이 놓여 있는 방향. (ex) (roll, pitch, yaw)로 표현
  - 3차원 회전을 표현하는 방법
    - Euler Angle: (roll, pitch, yaw)
      - Gimbal Lock (회전축이 겹치는) 문제 발생: 동일 자세를 나타내는 다수의 Euler Angle 조합 발생 (singularity)
      - 회전을 연속적으로 계산하거나 interpolation할때 불안정
    - Quaternion : $q = (w, x, y, z) = w + xi + yj + zk$ $(i, j, k$: 허수)
      - 3차원 회전을 4개의 수로 표현하는 방식







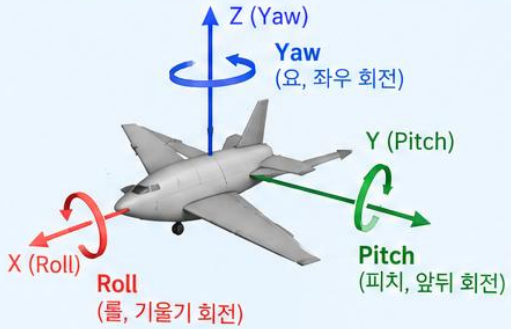
(참고) 3D 회전 표현: Euler Angle v.s. Quaternion

Euler Angle → Quaternion 변환 (직관적 설명)

Euler Angle(roll, pitch, yaw)로 표현된 3D 회전을 동일한 회전을 나타내는 Quaternion으로 변환하는 방법을 그림과 함께 단계적으로 설명합니다.

1 Euler Angle이란?

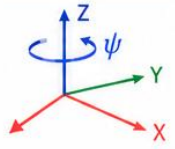
3D 물체의 회전을 3개의 각도로 표현하는 방법입니다.
가장 널리 쓰이는 순서는 Z-Y-X (yaw → pitch → roll) 입니다.



2 Z-Y-X 순서로 회전하는 과정

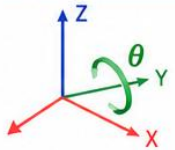
보통 항공기나 게임 엔진에서 많이 사용하는 순서입니다.
Yaw(ψ) → Pitch(θ) → Roll(ϕ)
순서로 회전한다고 가정합니다.

1. Yaw (Z축 회전)



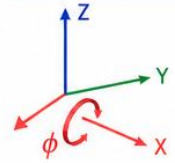
Z축을 중심으로 ψ만큼 회전
(좌우 방향 변경)

2. Pitch (Y축 회전)



회전된 상태에서 Y축을 중심으로 θ만큼 회전
(위아래 방향 변경)

3. Roll (X축 회전)



회전된 상태에서 X축을 중심으로 ϕ만큼 회전
(기울기 변경)

3 각각의 회전을 Quaternion으로 표현

각 축에 대한 회전을 개별 Quaternion으로 먼저 만듭니다.
(각도는 라디안 기준이며, 절반 각도 $\frac{\text{각도}}{2}$ 를 사용합니다)

Z축 회전 (Yaw, ψ)

$$q_z = \left(\cos\frac{\psi}{2};\ 0,\ 0,\ \sin\frac{\psi}{2}\right)$$

Y축 회전 (Pitch, θ)

$$q_y = \left(\cos\frac{\theta}{2}, 0, \sin\frac{\theta}{2}, 0\right)$$

X축 회전 (Roll, ϕ)

$$q_x = \left(\cos\frac{\phi}{2}, \sin\frac{\phi}{2}, 0, 0\right)$$

4 순서대로 곱해서 최종 Quaternion 구하기

Z → Y → X 순서로 회전하므로,
최종 Quaternion은 다음과 같이 곱합니다.

$$q = q_x \cdot q_y \cdot q_z$$

(Quaternion 곱셈은 순서가 중요합니다!)

5 계산을 전개하면 (Z-Y-X 순서 공식)

위의 곱셈을 전개하면, 최종 Quaternion $q = (w,\ x,\ y,\ z)$ 는 다음과 같습니다.

$$\begin{aligned} w &= c_r c_p c_y + s_r s_p s_y \\ x &= s_r c_p c_y - c_r s_p s_y \\ y &= c_r s_p c_y + s_r c_p s_y \\ z &= c_r c_p s_y - s_r s_p c_y \end{aligned}$$

$$\begin{aligned} c_r &= \cos\frac{\phi}{2};\quad s_r = \sin\frac{\phi}{2} \quad \text{(Roll)} \\ c_p &= \cos\frac{\theta}{2};\quad s_p = \sin\frac{\theta}{2} \quad \text{(Pitch)} \\ c_y &= \cos\frac{\psi}{2};\quad s_y = \sin\frac{\psi}{2} \quad \text{(Yaw)} \end{aligned}$$

6 예제: (Roll, Pitch, Yaw) = (30°, 20°, 45°)

각도를 라디안으로 변환하여 계산해봅니다.

1. 각도를 라디안으로 변환

$$\begin{aligned} \phi &= 30^\circ = \frac{\pi}{6} \\ \theta &= 20^\circ = \frac{\pi}{9} \quad (\approx 0.349) \\ \psi &= 45^\circ = \frac{\pi}{4} \end{aligned}$$

2. 각 절반의 삼각함수 값

$$\begin{aligned} c_r &= \cos 15^\circ = 0.9659 \\ s_r &= \sin 15^\circ = 0.2588 \\ c_p &= \cos 10^\circ = 0.9848 \\ s_p &= \sin 10^\circ = 0.1736 \\ c_y &= \cos 22.5^\circ = 0.9239 \\ s_y &= \sin 22.5^\circ = 0.3827 \end{aligned}$$

3. 최종 Quaternion

$q = (w,\ x,\ y,\ z)$

$$q \approx (0.8976,\ 0.2391,\ 0.1893,\ 0.3314)$$

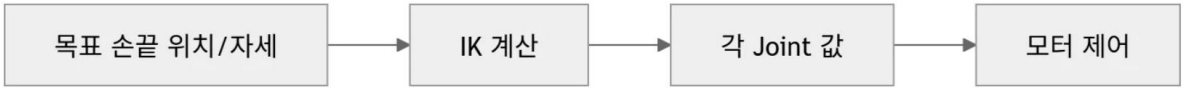
7 같은 회전을 시각적으로 비교

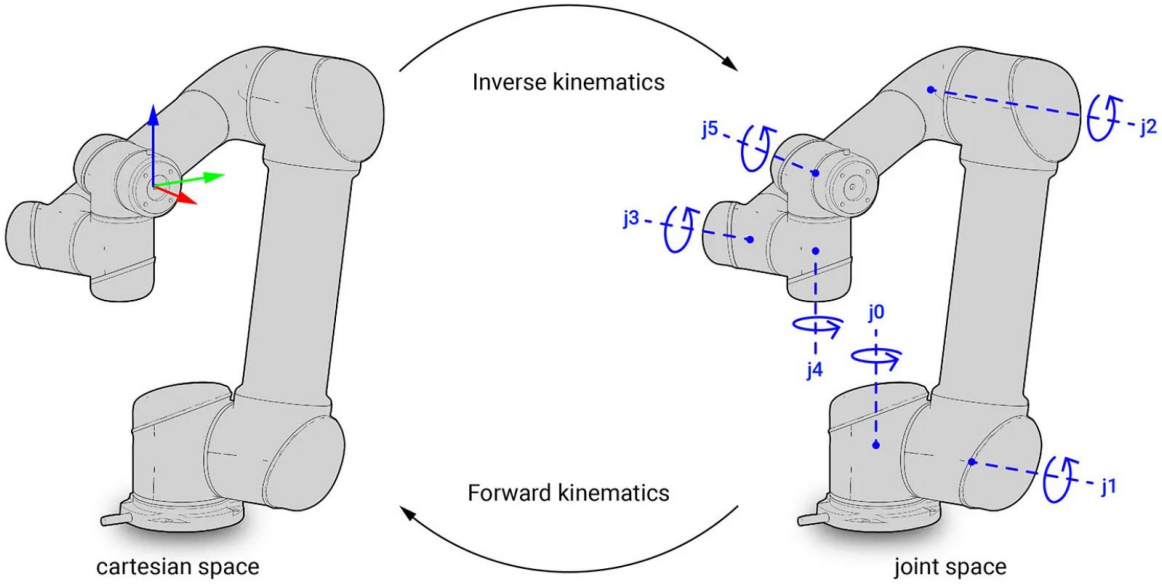
아래는 (30°, 20°, 45°) 회전을 Euler Angle과
해당 Quaternion으로 표현했을 때의 결과입니다.
두 표현은 동일한 3D 회전을 나타냅니다.



FK(Forward Kinematics), IF(Inverse Kinematics)

- end-effector의 위치와 joint 값의 관계
  - Forward Kinematics (FK)
    - Joint 값 → EE의 위치와 자세
  - Inverse Kinematics (IK)
    - EE의 위치와 자세 → joint 값
    - 방법: 기하학적 계산법, 해석적 계산법, 수치해석적 계산법, 딥러닝(?)
      - 경우에 따라 해가 0/1/다수 일 수 있음





(참고) 3D 회전 표현: Euler Angle v.s. Quaternion

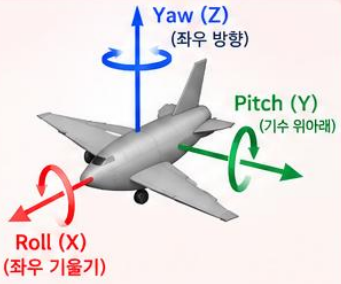
Euler Angle vs Quaternion: 짐벌락과 보간(Interpolation) 관점 비교

같은 3D 회전을 표현하지만, 두 방식은 '어떤 문제가 있고, 어떻게 움직임을 보간하느냐'에서 큰 차이가 납니다.

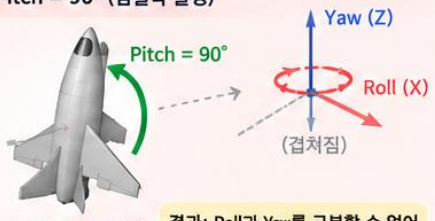
1 짐벌락 (Gimbal Lock) 비교

Euler Angle: 특정 자세에서 두 축이 겹쳐 자유도(1 DOF)를 잃음

1 정상 상태 (세 축이 독립적)



2 Pitch = 90° (짐벌락 발생)



이 상태에서 Roll 축과 Yaw 축이 같은 방향으로 겹침!

결과: Roll과 Yaw를 구분할 수 없어 1개의 회전 자유도를 잃음 (3 DOF → 2 DOF)

Quaternion: 어떤 자세에서도 축이 겹치지 않아 짐벌락이 없음



단위 Quaternion은 4차원 공간의 단위 구(S^3) 위의 한 점으로 3D 회전을 표현합니다.

단위 구 표면 위에서 연속적으로 이동할 뿐, 특이점이 존재하지 않습니다.

어떤 방향을 향하더라도 축이 겹치는 상황이 없으므로 짐벌락이 발생하지 않습니다.

2 Interpolation (보간) 비교

Euler Angle 보간: 각도를 각각 선형 보간 (LERP)

Roll, Pitch, Yaw 각도를 독립적으로 선형 보간하면,
각 축이 따로 움직이면서 중간 경로가 비직관적이고 부자연스러울 수 있습니다.



Quaternion 보간: SLERP (구면 선형 보간)

단위 구(S^3) 위에서 두 회전 사이의 최단 경로를 따라 일정한 속도로 부드럽게 보간합니다.
항상 일정한 각속도(constant angular velocity)로 자연스러운 회전이 가능합니다.



3 수치적/개념적 이유

Euler Angle

- 3개의 각도를 순서대로 적용 (Euler Sequence)
- 순서(Z–Y–X 등)에 따라 결과가 달라짐
- 특정 자세에서 축이 겹치는 특이점 발생 (짐벌락)
- 각도를 각각 보간하면 중간 경로가 비직관적일 수 있음
- 직관적으로 이해하기 쉽고, 사람에게는 편리한 표현

Quaternion 쿼터락

- 하나의 회전축 + 하나의 회전각을 4개의 수로 표현
- 어떤 자세에서도 특이점이 없음
- SLERP로 최단 경로, 일정한 속도, 자연스러운 보간 가능
- 여러 회전을 곱셈 하나로 합성 가능
- 수학적으로 조금 더 복잡하지만, 3D 회전 계산에 매우 강력

4 한눈에 비교

항목	Euler Angle	Quaternion
회전 표현 방식	(roll, pitch, yaw) 3개의 각도	(w, x, y, z) 4개의 수 (단위 quaternion)
짐벌락	발생함 (특정 자세에서 1 DOF 손실)	발생하지 않음
보간 방법	각도별 선형 보간 (LERP)	구면 선형 보간 (SLERP)
중간 경로	비직관적일 수 있고, 부자연스러움	최단 경로, 부드럽고 자연스러움
회전 합성	순서에 민감 (Z–Y–X 등)	Quaternion 곱셈으로 간단
직관성 (사람에게)	이해하기 쉬움	직관적이지 않지만 매우 강력함
주요 사용처	UI, 간단한 입력, 디버깅용	게임, 그래픽스, 로봇공학, 드론, 애니메이션 등

Action-Token 생성 방법 (Transformer 계열)

- Step 1.
  - 어떤 값을 action으로 사용할 것인가? Join space v.s. Cartesian space
- Step 2.
  - 각 차원의 (의미 있는 제어) 범위 지정: $\Delta_x \in [l, u]$
    - 방법1: 고정된 물리적 범위 사용
    - 방법2: 데이터 분포를 기준으로 범위 지정
- Step 3.
  - 연속 값을 bin으로 구분
  - Bin width = $(u - l)/n_{bin}$ (n_{bin}: bin의 수. Ex. $n_{bin} = 256$)
- Step 4.
  - 실제 값을 bin index로 변환 (discretization)
- Step 5. tokenization
  - Bin index를 실제 값의 token으로 사용
    - Action의 각 element (ex. $(x, y, z, roll, pitch, yaw, gripper)$)를 독립적으로 토큰화
- Step 6. de-tokenization
  - (VLA) 모델이 예측한 token을 다시 연속값으로 복원 (ex. 예측 토큰(=bin)의 중간값)

VLA Output and Robot Control

- Typical VLA Output Format
  - Joint Space Action
    - Ex) 각 관절의 목표 각도, 각속도, 변화량 등
  - Cartesian Space Action
    - Ex) end effector의 위치 (변화량), 각속도 (변화량)
- Robot Control Layer
  - VLA output을 로봇 모터에 입력되는 전류/토크 명령으로 변환
  - Output이 joint space action인 경우
    - 출력 값의 의미 (절대값, 변화량, 등)에 따라 목표 joint 값 계산
    - 로봇 저수준 제어기가 주어진 목표 joint 값에 따라 모터를 구동
  - Output이 Cartesian space action인 경우
    - 출력 값의 의미 (절대값, 변화량, 등)에 따라 end effector의 목표 pose 계산
    - IK를 이용해 목표 pose를 만족하는 joint 값 계산
    - 계산된 joint 값에 따라 모터 구동

대표적인 VLA Outputs

- Cartesian VLA

  VLA → EE Pose/Delta → IK(Inverse Kinematics) / Controller → Joint command → Robot

- Joint-action VLA

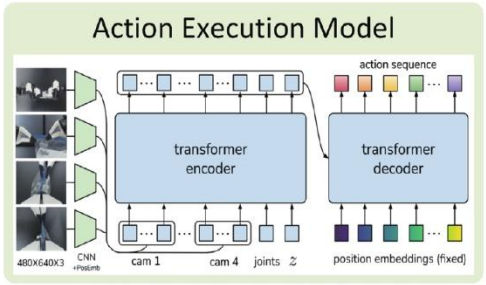
  VLA → Joint command → Controller → Robot

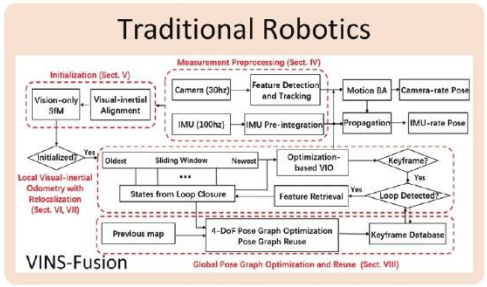
	EE Cartesian Action	Joint Action
출력 의미	손(EE)을 어디로 움직일지	관절을 어떻게 움직일지
예	EE의 3D 좌표 (x,y,z), 회전(roll, pitch, yaw), gripper 제어: [$\Delta x, \Delta y, \Delta z$, Δroll, Δpitch, Δyaw, gripper]	각 관절값의 모터 제어값+gripper 제어: $\Delta q_1, \Delta q_2, \dots, \Delta q_n$
직관성	높음	낮음
로봇 간 일반화	상대적으로 유리	불리
embodiment 의존성	낮은 편	높음
IK 필요	필요	불필요
저수준 제어 능력 (속도/정밀도)	제한적 (로봇내부의 IK 계산을 거쳐야 하므로 반응 지연 및 singularity 위험 있음)	높음
정교한 전신동작	어려울 수 있음	유리
redundancy 활용	IK/controller가 담당	모델이 직접 활용 가능 (같은 EE pose를 만드는 관절 자세가 여러 개 존재할 수 있음 ← 모델이 어떤 관절 자세를 사용할지 "직접" 결정 → 장애물 회피, 자연스러운 동작 제어 등에 유리)
학습 난이도	상대적으로 낮음	상대적으로 높음 (로봇의 현재 자세에 따라 같은 행동에 필요한 관절 값이 달라 짐)
데이터 수집	사람의 손 비디오나 가상 환경 데이터 변환이 용이	실제 로봇의 원격 제어 및 모터의 로그 데이터 필요

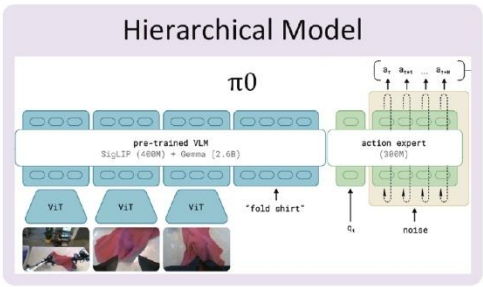
출력 종류	예시	특징
연속 제어값	[Δx, Δy, Δz, Δroll, Δpitch, Δyaw, gripper]	로봇 팔을 직접 제어하기 좋음
절대 Pose	[x, y, z, roll, pitch, yaw]	목표 end-effector 위치/자세 출력
Joint Action	[q1, q2, ..., qN] 또는 joint velocity	관절 위치/속도를 직접 출력
Discrete Action	MOVE_LEFT, GRASP, STOP	행동 공간을 몇 개의 클래스로 이산화
Action Token	<x_32> <y_18> <z_07> <grip_close>	행동을 토큰화하여 LLM처럼 autoregressive하게 생성
Action Chunk	a_t, a_{t+1}, ..., a_{t+H}	한 번에 미래 여러 timestep의 행동 출력
고수준 Skill/Command	pick(object), place(location)	실제 trajectory는 별도 controller/planner가 생성

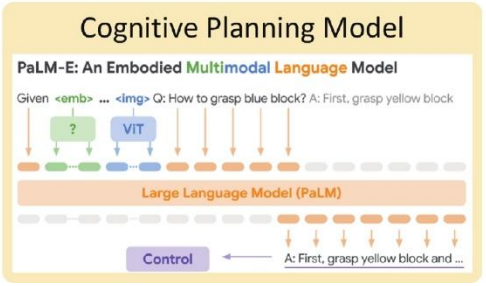
VLA Models

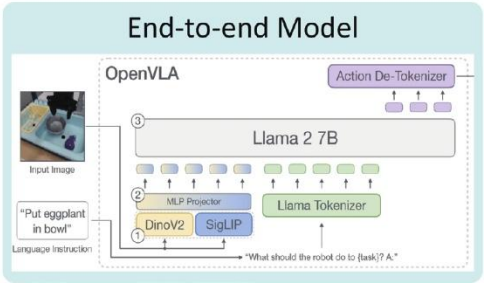
VLA Timeline











1996 PRM

1998 RRT

2011 RRT*

2018 VINS-Fusion, MSCKF-VIO

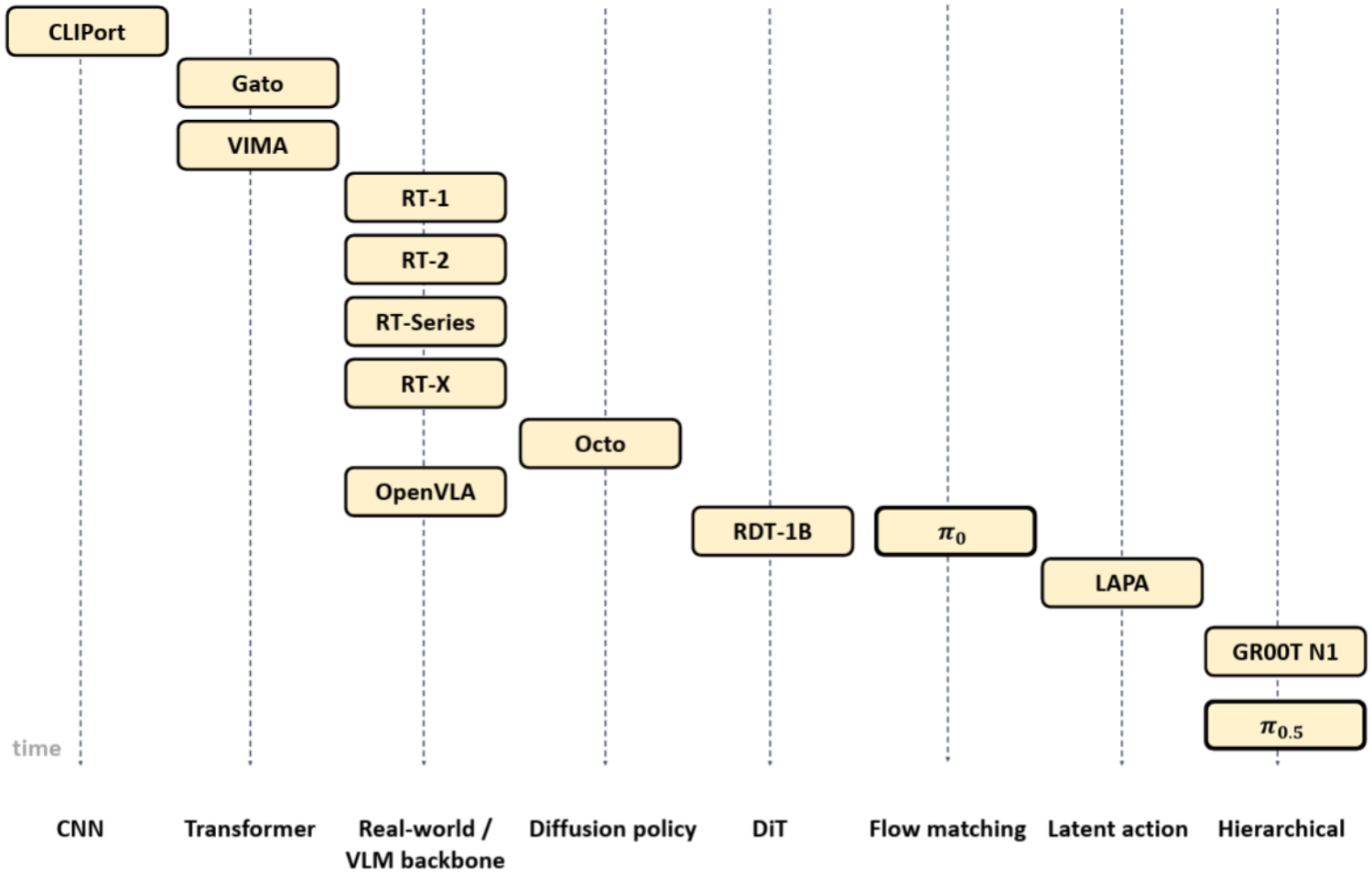
2022 RT-1, SayCan, ChatGPT for Robotics, PaLM-E

2023 RT-2, Diffusion Policy, ALOHA, RT-X, Octo, OpenVLA, π0, Hybrid-VLA

2024 GO-1, CoT-VLA, Helix

2025 GR00T N1

Timeline of VLA Models



VLA Timeline

시기	모델	기관	핵심 키워드
2022.05	Gato	DeepMind	단일 Transformer로 다양한 작업을 수행하는 generalist agent
2022.12	RT-1	Google	대규모 실로봇 데이터 기반 Robotics Transformer
2023.04	ACT	Stanford (ALOHA)	Action Chunking과 저비용 양팔 조작
2023.07	RT-2	Google DeepMind	VLM을 로봇 제어에 직접 연결한 Vision-Language-Action 모델
2024.06	OpenVLA	Stanford 외	오픈소스 7B VLA, Open X-Embodiment 데이터 활용
2024.10	π_0	Physical Intelligence	Flow Matching 기반 고주파수의 연속 액션 생성
2025.02	Figure-Helix	Figure AI	휴머노이드를 위한 이중 시스템(System 1/2) 구조
2025.03	Groot N1	NVIDIA	휴머노이드 파운데이션 모델, 이중 시스템 구조
2025.03	Gemini Robotics 1.0	Google DeepMind	Gemini 2.0 기반 VLA와 Embodied Reasoning
2025.04	$\pi_{0.5}$	Physical Intelligence	이종 데이터 co-training을 통한 open-world 일반화
2025.05	Groot N1.5	NVIDIA	합성 데이터 확장과 일반화 성능 개선
2025.06	SmolVLA	Hugging Face	소형(450M) 오픈소스 VLA, 커뮤니티 데이터 학습
2025.09	WALL-OSS	X Square Robot	오픈소스 VLA, embodied reasoning과 액션의 통합
2025.09	Gemini Robotics 1.5	Google DeepMind	사고(thinking) 능력과 cross-embodiment 학습
2025.11	$\pi_{0.6}$	Physical Intelligence	실사용 경험(RL)을 통한 성능 개선

RT-1

https://robotics-transformer1.github.io/

RT-1

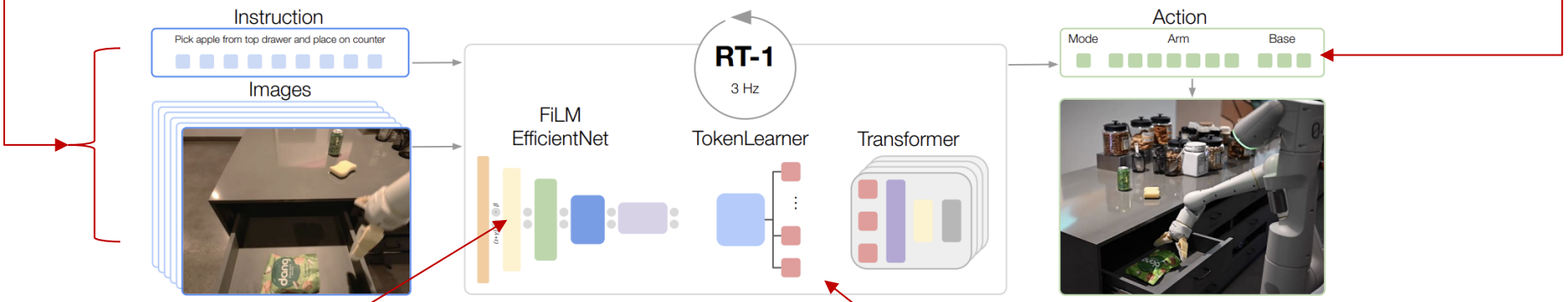
- 로봇 학습이 task-specific 방식에서 generalist policy로 넘어가는 과정에서 등장한 중요한 전환점
- 기존 로봇 학습 방법들 문제점
  - 특정 작업을 위한 데이터를 모아 특정 작업용 정책을 학습
    - 새로운 작업이나 새로운 환경에 적용 어려움 (new tasks, objects, background, instruction set)
- LLM, VLM
  - 대규모 데이터와 고용량 모델이 범용성을 크게 높임
- RT-1
  - 로봇에서도 하나의 큰 모델이 여러 작업을 학습하고 일반화할 수 있는가?
    - 로봇에서도 범용 backbone model이 가능할까?
    - 대규모 multi-task 데이터가 generalization을 높일 수 있을까?
    - 그런 모델을 실제 제어에 쓸 수 있을 만큼 효율적으로 만들 수 있을까?
      - Control Frequency: 3Hz (RT-1이 다루는 manipulation task에 실용적으로 적응 가능한 속도)
      - 한번의 제어 loop (센서 캡처, 전처리, 명령 전달, 로봇 적용 지연 등을 모두 포함)는 333ms 이므로 inference latency budget을 100ms 이하로 가정.

RT-1

Inputs: 6 Images history + 자연어 instruction

Output: Tokenized action (256 bins)

- 7-arm movements: x, y, z, roll, pitch, yaw, gripper opening
- 3-base movement: x, y, z, yaw
- 1-mode: arm제어, base 제어, episode 종료 중 하나



(a) RT-1 takes images and natural language instructions and outputs discretized base and arm actions. Despite its size (35M parameters), it does this at 3 Hz, due to its efficient yet high-capacity architecture: a FiLM (Perez et al., 2018) conditioned EfficientNet (Tan & Le, 2019), a TokenLearner (Ryoo et al., 2021), and a Transformer (Vaswani et al., 2017).

❖ Image Encoding: EfficientNet-B3

- 6개 각각의 image를 $9 \times 9 \times 512$ 크기의 feature map으로 변형 후 펼쳐서 81개의 visual token 생성

❖ Universal Sentence Encoder + FiLM

- Universal Sentence Encoder: Instruction 을 512-d embedding vector로 변형
- FiLM layer: embedding된 instruction을 EfficientNet-B3 중간 중간에 삽입 (→ task relevant visual feature extraction)

❖ Token Learner

- Vison-Language token (81개) 중 Task와 관계가 큰 중요한 token만을 추출 (8개)
- ← transformer 계산량 축소를 통해 빠른 추론 (로봇 동작 결정)

❖ (Decoder Only) Transformer

- **입력: 이미지 토큰 (6장*8개/장=48개) + Positional Encoding**

학습 방식

- **Loss: categorical (i.e., bin) cross-entropy (i.e., classification 문제)**
- **연속적인 각 action 값을 256개의 bin으로 나눈 후 Action 출력의 각 element들이 속할 bin index를 학습**

RT-1

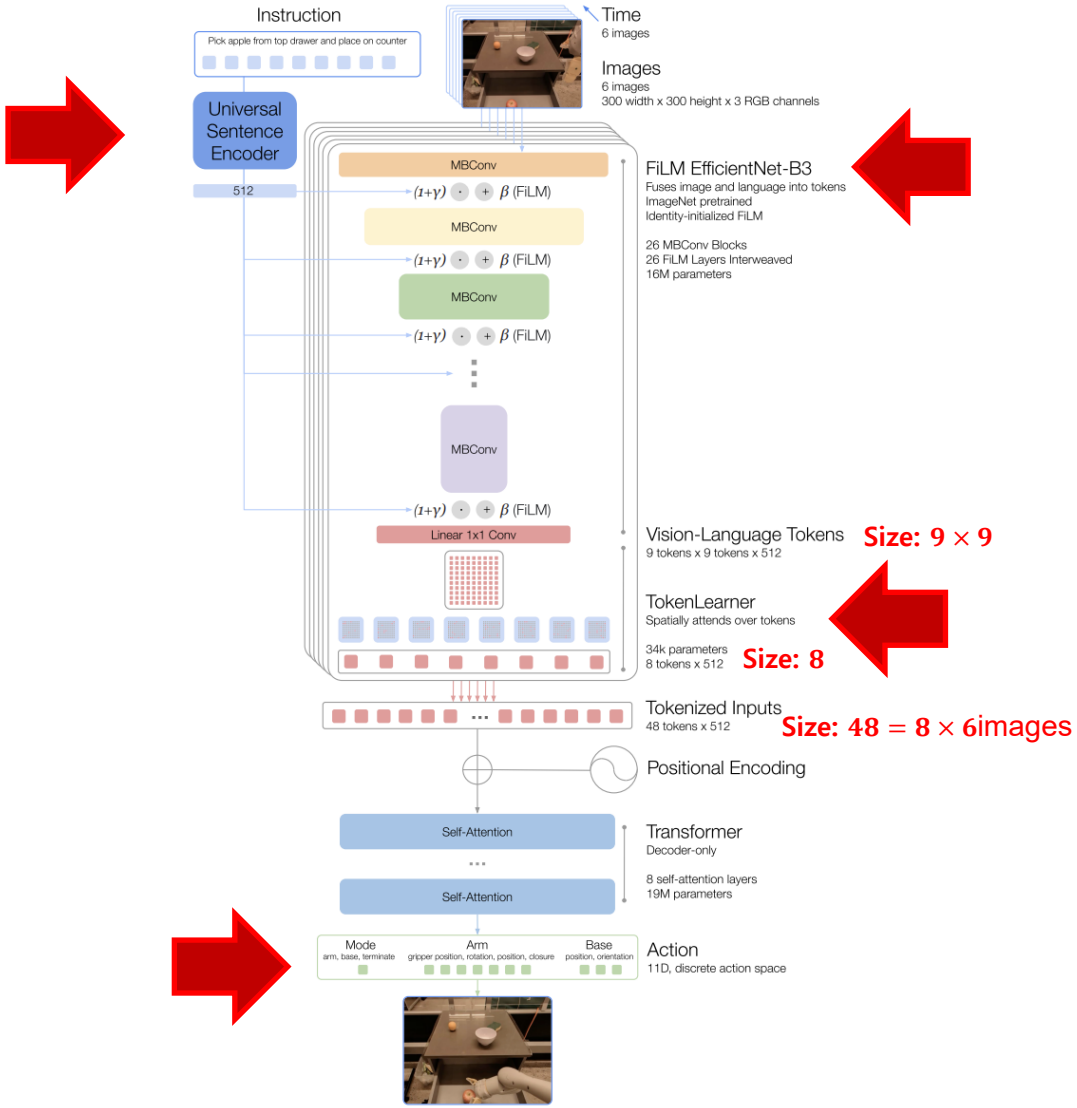


Figure 3: The architecture diagram of RT-1. The instruction is transformed into a USE embedding and used to condition a pre-trained EfficientNet via FiLM layers. The resulting vision-language tokens are reduced by the TokenLearner and fed into a decoder-only Transformer, which outputs tokenized actions.

RT-1

- 학습 데이터
  - 주안점
    - Scale and Breadth
    - Task들 사이의 구조적 연결성
  - **13**대의 로봇 fleet을 사용해 **17**개월 동안 약 **130K** demonstration을 수집
  - 수행 가능한 instruction 수: **700**개 이상
  - Diversity
    - Environment (place, background, etc.)
    - Objects
    - Skills: skills were chosen to demonstrate multiple behaviors with many objects

    → instruction generalization과 object generalization의 기반

"데이터 안의 task들이 서로 충분히 연결되어 있어야, 모델이 구조적으로 비슷한 패턴을 발견하고 새로운 조합으로 일반화 할 수 있음 (skill + object → 새로운 instruction으로 확장)"

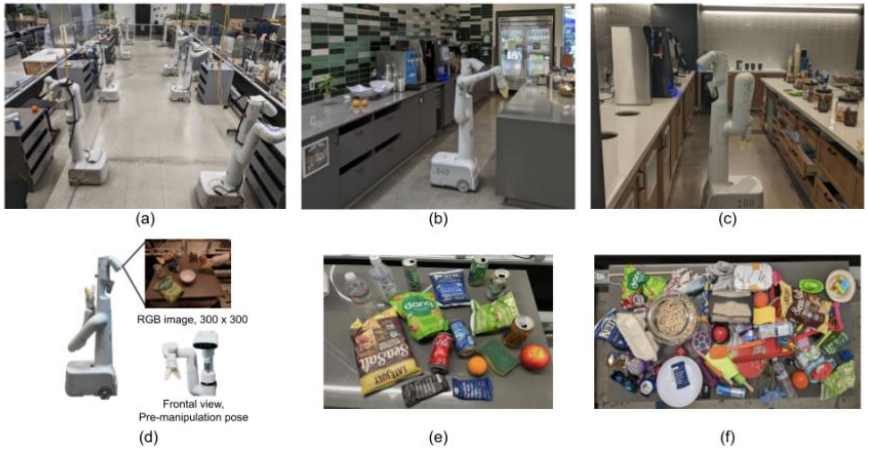


Figure 2: (a) Robot classroom where we collect data at scale; (b) a real office kitchen, one of the two realistic environments used for evaluation (named Kitchen1 in the rest of the paper); (c) a different office kitchen used for evaluation (named Kitchen2 in the rest of the paper); (d) mobile manipulator used throughout the paper; (e) a set of objects used for most of the skills to expand skill diversity; (f) a more diverse set of objects used mostly to expand object diversity of the picking skill.

Skill	Count	Description	Example Instruction
Pick `Object`	130	Lift the object off the surface	pick iced tea can
Move `Object` Near `Object`	337	Move the first object near the second	move pepsi can near rxbar blueberry
Place `Object` Upright	8	Place an elongated object upright	place water bottle upright
Knock `Object` Over	8	Knock an elongated object over	knock redbull can over
Open `Drawer`	3	Open any of the cabinet drawers	open the top drawer
Close `Drawer`	3	Close any of the cabinet drawers	close the middle drawer
Place `Object` into `Receptacle`	84	Place an object into a receptacle	place brown chip bag into white bowl
Pick `Object` from `Receptacle` and Place on the Counter	162	Pick an object up from a location and then place it on the counter	pick green jalapeno chip bag from paper bowl and place on counter
Section 6.3 and 6.4 tasks	9	Skills trained for realistic, long instructions	open the large glass jar of pistachios pull napkin out of dispenser grab scooper
Total	744		

Table 1: The list of skills collected for RT-1 together with their descriptions and example instructions.

RT-1

- 실제 환경에서 3000회 이상의 real-world evaluation을 수행
- 평가 기준
  - Seen task performance
    - 학습 데이터에 포함된 instruction을 대상
    - object placement, time of day, robot position 같은 조건이 달라질 수 있음
  - Unseen task generalization
    - 목적
      - 단순 암기형 정책이 아니라, 기존 패턴을 재조합해 새로운 task로 확장할 수 있는지를 평가
    - 21개의 새로운 instruction 수행을 평가
    - 새로운 instruction
      - 완전히 낯선 물체나 전혀 새로운 skill이 아니라,
      - 학습 중 보았던 object와 skill을 새로운 방식으로 조합한 것
      - Ex) skill “pick”, object “apple” → instruction “pick up the apple” 학습에서 제외
  - Robustness
    - Distractor robustness (30개)
      - 목표 물체 주변에 다른 물체가 많을 때도 제대로 수행할 수 있는지를 보는 평가
    - Background robustness (22개)
      - 주방 환경 자체가 달라졌을 때, 예를 들어 조명, 배경, countertop 표면이 바뀌었을 때도 잘 동작하는지를 평가
  - Long-horizon scenarios



Figure 4: Evaluation scenarios for distractors (first row), from left to right: easy (0-5 distractors), medium (9 distractors), hard (9 distractors and occluded object); background (second row), from left to right: original environment, patterned table cloth, new kitchen; and realistic scenarios in the real kitchen (third row), generalization levels from left to right: $L1$, $L2$ and $L3$.

RT-1

❑ RT-1 설계 요소들이 성능에 미치는 영향 (Ablation)

Discrete Action Representation (256 bins)

- w/continuous actions: 성능 저하
- complex multi-modal action distribution 표현 능력 차이 (Gaussian continuous action은 하나의 mode만 잘 표현하기 때문에 더 복잡한 demonstration data에 부적합)

ImageNet Pretraiing

- w/o pre-training: 성능 저하
- large and diverse visuals of the ImageNet dataset 덕분에 일반화와 robustness가 크게 향상

→ 이미 넓은 시각 패턴을 학습한 backbone 위에 로봇 task를 올리는 편이 훨씬 유리

			Distractors				Backgrounds	
Model	Seen Tasks	Unseen Tasks	All	Easy	Medium	Hard	All	Inference Time (ms)
Gato (Reed et al., 2022)	65 (-32)	52 (-24)	43 (-40)	71	44	29	35 (-24)	129
BC-Z (Jang et al., 2021)	72 (-25)	19 (-57)	47 (-36)	100	67	7	41 (-18)	5.3
BC-Z XL	56 (-41)	43 (-33)	23 (-60)	57	33	0	35 (-24)	5.9
RT-1 (ours)	**97**	**76**	**83**	100	100	64	**59**	15
RT-1 w/o big model	89 (-8)	62 (-14)	77 (-6)	100	100	50	53 (-6)	13.5
RT-1 w/o pre-training	84 (-13)	43 (-33)	60 (-23)	100	67	36	41 (-18)	15
RT-1 w/ continuous actions	68 (-29)	43 (-33)	37 (-46)	71	67	0	35 (-24)	16
RT-1 w/ auto-regressive actions	85 (-12)	71 (-5)	67 (-16)	100	78	43	**65 (+6)**	36
RT-1 w/o history	82 (-15)	62 (-14)	50 (-33)	71	89	14	**59 (+0)**	15
RT-1 w/o Transformer	86 (-13)	62 (-14)	67 (-16)	100	100	29	**59 (+0)**	26

Observation History (6 images)

- w/o history: 성능 저하
- 로봇 제어에서는 현재 프레임 하나보다 짧은 시간 맥락이 중요

Model Size and Capacity

- w/o big model: 성능 저하 (35M → 21M)
- 어느 정도 충분한 capacity를 가져야 다양한 task와 instruction을 흡수할 수 있다

(Decoder Only) Transformer

- w/o transformer: 성능 저하 (transformer 제거 후 pre-trained EfficientNet만 사용
- 시각 backbone만 좋아서는 충분하지 않고, 여러 시점의 token과 instruction-conditioned 정보를 sequence로 통합해서 action을 예측하는 구조가 효율적 (esp. unseen and distractor)

Auto-Regressive Action

- w auto-regressive actions: 성능 저하 (esp. inference time 2배 증가)
- 성능을 크게 개선하지 않으면서 가장 큰 slowdown을 일으킴

RT-1. 한계점

- Imitation Learning
  - **성공한** demonstration을 보고 그 행동을 따라 배우는 BC 방식
  - demonstration에 없는 새로운 전략을 스스로 탐색하거나, **자율적**으로 시행착오를 통해 더 나은 행동을 발견하는 구조는 아님
- Compositional Generalization
  - 본 적 있는 skill과 object를 새로운 조합으로 묶는 compositional generalization
  - 완전히 새로운 물리 상호작용이나 전혀 새로운 작업 개념을 스스로 학습한 것은 아님
- Planning의 외부 의존성
  - 긴 작업에서도 좋은 성능을 보였지만 그 결과는 SayCan 같은 상위 planning framework와 결합한 경우임
  - 여러 단계의 복잡한 계획을 스스로 세우는 모델이라기보다, 주어진 하위 skill을 안정적으로 수행하는 low-level executor (실행 성능 위주)
- Embodiment Generalization 제약
  - heterogeneous data 결과는 "다른 데이터도 어느 정도 흡수할 수 있다"는 가능성을 보여줌
  - 그러나, action space와 morphology가 다른 로봇까지 완전히 자연스럽게 일반화한 단계는 아님
- 대규모 데이터 의존성
  - large-scale real-world data: 13대의 로봇으로 17개월 동안 약 130k demonstration을 수집
  - "데이터를 충분히 모으면 가능하다"는 점을 보여주었지만, 그 데이터 수집 비용 자체는 여전히 큰 장벽
- Robustness
  - 실험 결과: background robustness는 59 수준으로, seen task 97과 비교하면 아직 격차가 큼

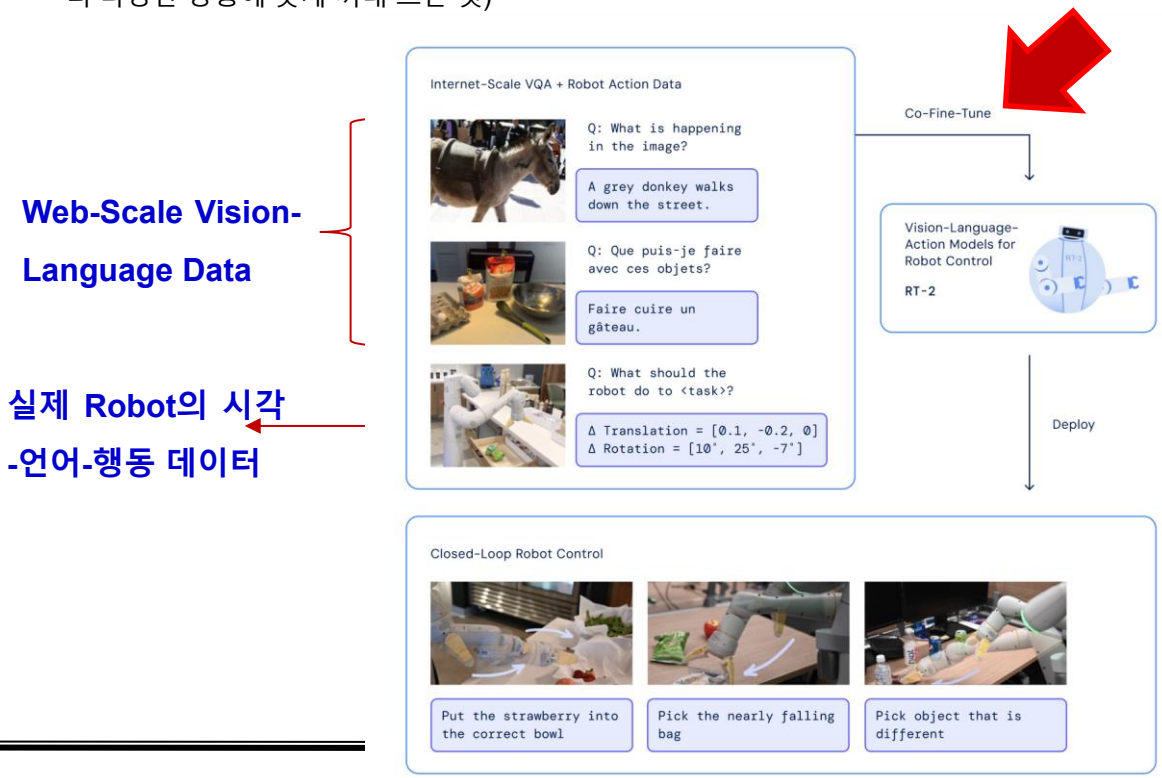
RT-2

https://robotics-transformer2.github.io/

https://arxiv.org/abs/2307.15818

RT-2

- RT-1
  - 주로 로봇 데이터 안에서 시각 입력과 언어 명령을 행동으로 연결하는 데 초점
- RT-2
  - RT-1에 Web-Scale Vision-Language Pre-training이 가진 Semantic 이해와 추론 능력을 직접 연결하려는 시도
  - 웹에서 배운 지식을 그대로 로봇 skill로 바꾸는 것은 아님
    - 집기, 놓기, 이동 같은 물리적 동작 자체는 여전히 로봇 데이터에서 학습
    - 웹 데이터에서 학습한 것은 동작을 언제, 어디에, 어떤 대상에 써야 하는지를 더 넓게 이해하는 능력임 (이미 배운 동작을 더 다양한 상황에 맞게 꺼내 쓰는 것)



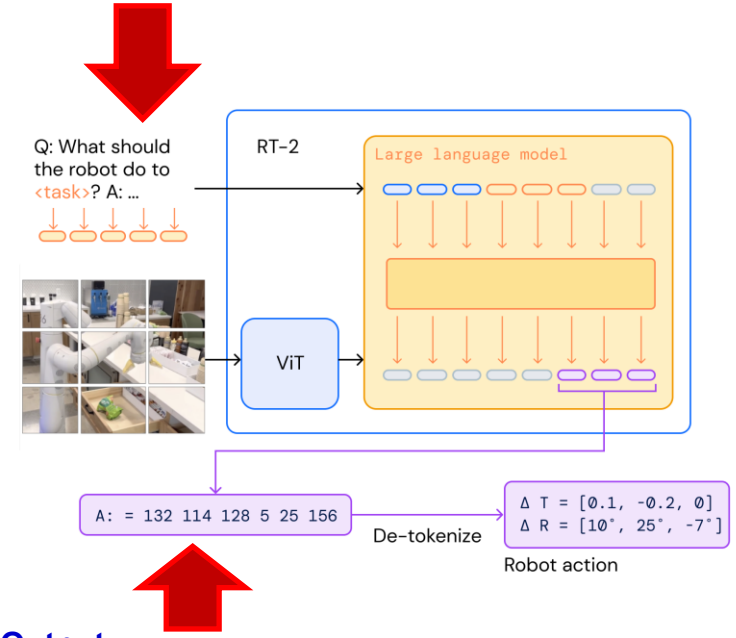
RT-2. Model

- Backbone
  - 기존 VLM (PaLI-X와 PaLM-E)에 로봇 제어를 위한 action token만 추가
    - RT-2-PaLI-X: 5B, 55B, RT-2-PaLM-E: 12B
  - Backbone별 tokenizer가 다르므로 action token 추가 방식만 상이함.
- Co-Fine-Tuning
  - Training with robot data + web scale vision-language data
  - 학습 시에는 배치 내에서 robot data의 sampling weight를 높여 두 데이터의 비율을 조정
    - RT-2-PaLI-X: robotics data를 약 50% 비중으로 포함
    - RT-2-PaLM-E: robotics data를 약 66% 비중으로 포함
  - 실험 결과 robot-only fine-tuning보다 co-fine-tuning이 일반화 성능 우수
    - since the policies are exposed to both abstract visual concepts from web scale data and low level robot actions during fine-tuning
    - 사전학습된 VLM을 로봇 데이터만으로 미세조정할 경우, 기존에 학습한 시각적·시맨틱 표현이 약화 (forget)되어 성능 저하 발생
- Output Constraint
  - 추론: task에 따라 출력 가능한 토큰을 제한하여 실행 불가능한 출력 방지
    - 로봇 제어 task: action token만 생성 (constrain its output vocabulary via only sampling valid action tokens when the model is prompted with a robot-action task)
    - 일반 vision-language task: 자연어 전체 생성 가능
  - 대형 모델을 실제 제어에 연결하기 위해 클라우드 기반 추론 구조 사용
    - 즉, 로봇은 네트워크를 통해 action을 받아 제어에 사용
    - 55B model: 1-3Hz, 5B model: 5Hz



< robot actions as text strings >

Inputs:
- robot camera image,
- Text description (VQA) format



Output:
- String of numbers (i.e., tokens representing a robot action)

RT.2 Experiments

- 실험 목표: real-world generalization and emergent capabilities of RT-2
  1. How does RT-2 perform on seen tasks and more importantly, *generalize* over new objects, backgrounds, and environments?
  2. Can we observe and measure any *emergent capabilities* of RT-2?
  3. How does the generalization vary with *parameter count* and other *design decisions*?
  4. Can RT-2 exhibit signs of *chain-of-thought* reasoning similarly to vision-language models?
- 실험 설정
  - 6,000개의 평가 trajectory를 사용하며, 7DoF mobile manipulator 환경
  - 학습 데이터
    - Web scale data
      - PaLI-X: On Scaling up a Multilingual Vision and Language Model (https://arxiv.org/abs/2305.18565)
    - Robot demonstration data (RT-1)

❑ 일반화 성능

- seen task에서는 RT-1과 큰 차이가 없지만 unseen 조건에서는 성능 차이
- 웹 사전학습 기반 표현이 특정 환경에 국한되지 않고 긍정적으로 작동

4.1. How does RT-2 perform on seen tasks and more importantly, generalize over new objects, backgrounds, and environments?

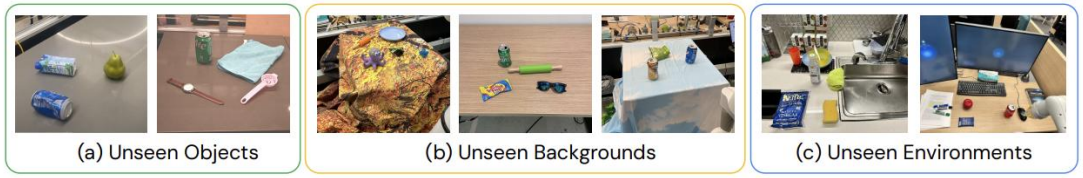


Figure 3 | Example generalization scenarios used for evaluation in Figures 4 and 6b and Tables 4 and 6.

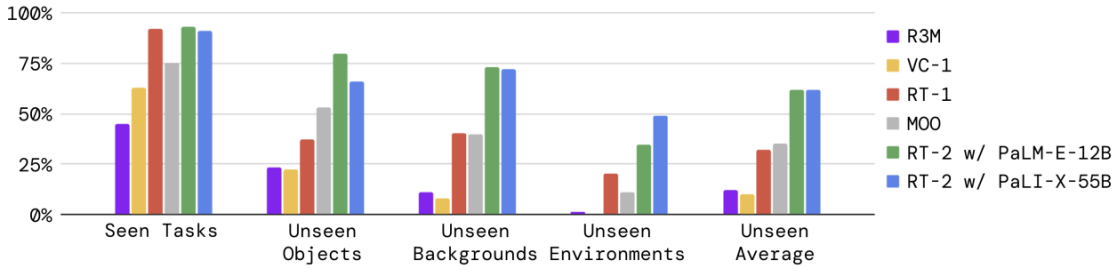


Figure 4 | Overall performance of two instantiations of RT-2 and baselines across seen training tasks as well as unseen evaluations measuring generalization to novel objects, novel backgrounds, and novel environments. Appendix Table 4 details the full results.

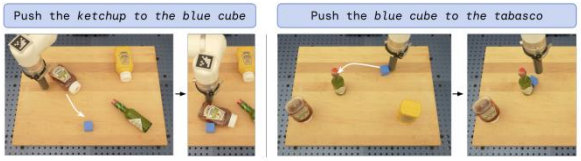


Figure 5 | Real-world out-of-distribution behaviors in the Language Table environment. Identical RT-2-PaLI-3B model checkpoint is used as in Tab. 1.

Model	Language-Table
BC-Zero (Jang et al., 2021)	72 ± 3
RT-1 (Brohan et al., 2022)	74 ± 13
LAVA (Lynch et al., 2022)	77 ± 4
RT-2-PaLI-3B (ours)	**90 ± 10**

Table 1 | Performance on the simulated Language-Table tasks (Lynch and Sermanet, 2020).

❑ Emergent Capability

- RT-2는 단순한 일반화를 넘어서 웹에서 학습한 개념을 행동으로 연결
  - 새로운 동작을 배우는 것이 아니라, 기존 동작을 *더 넓은 개념과 연결해 사용*
- 새로운 행동을 만들어낸 것이 아니라, ***행동을 선택하는 기준 자체를 확장***
  - 기존 (위치 기반) → RT-2 (개념 기반)
- Symbol Understanding
  - 추상적 기호 (숫자, 문자, 아이콘)이 무엇을 의미하는지 이해하고, 그 위치를 장면 안에서 찾아 행동으로 연결
  - Move coke can to X
- Reasoning
  - 관계와 의미 (수학적 관계, 의미 기반 관계, 관계 이해, 다국어 표현)를 기반으로 한 선택 능력
  - Move banana to the sum of two plus one
  - Pick a healthy drink
  - Move apple to cup with same color
- Human Recognition
  - Move coke can to Tayler Swift
  - Person with glasses

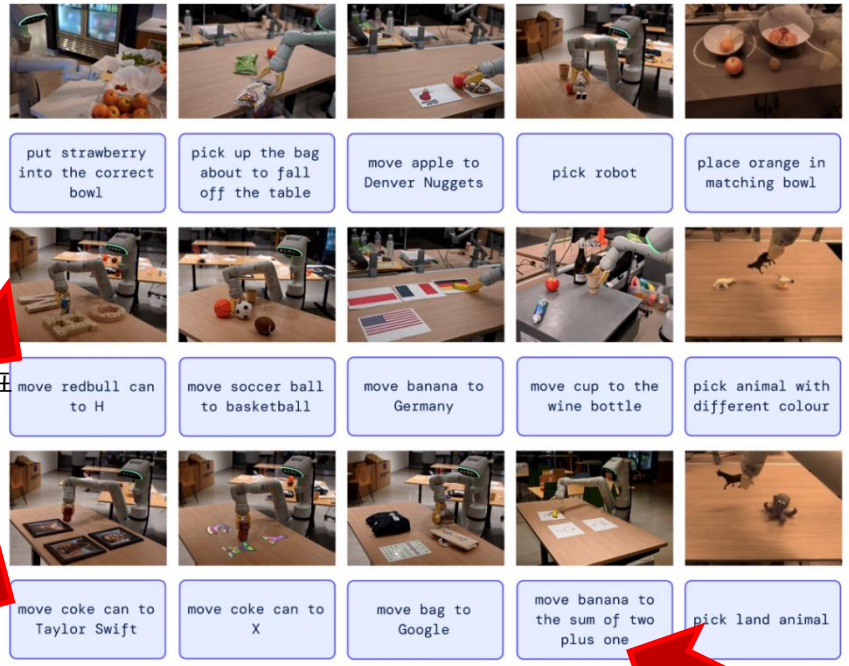
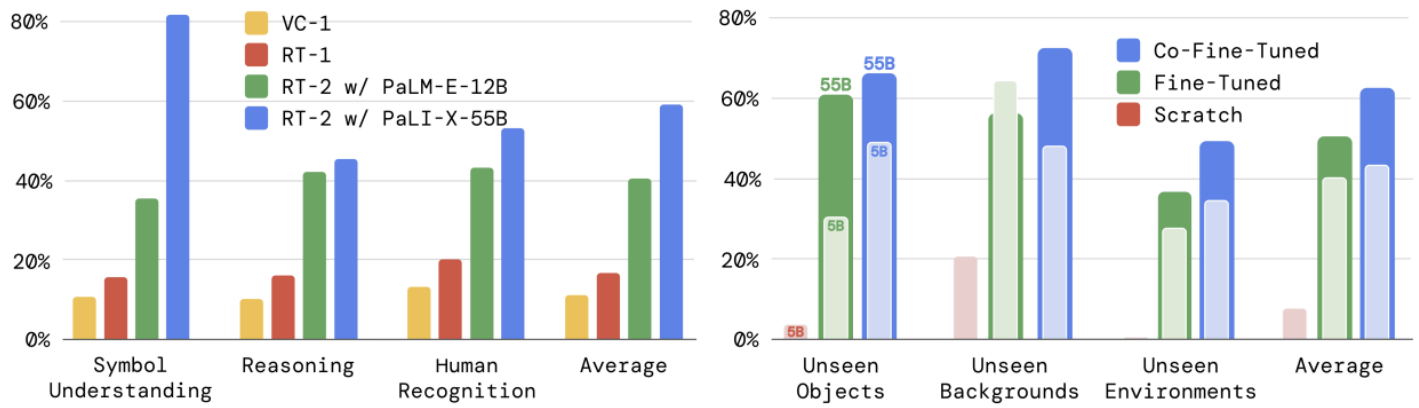


Figure 2 | RT-2 is able to generalize to a variety of real-world situations that require reasoning, symbol understanding, and human recognition. We study these challenging scenarios in detail in Section 4.

❑ 모델 크기와 학습 방식

- Scratch (큰 모델을 Pre-Train 없이) 학습은 잘 안 됨
- *Pre-Trained* 모델을 로봇 데이터만으로 fine-tuning하는 것보다, 웹 데이터와 로봇 데이터를 함께 쓰는 *co-fine-tuning*이 일반화 우수
  - 기존 VLM 학습에서 얻은 개념을 잊지 않게 해주는 것이 일반화 성능 향상



(a) Performance comparison on various emergent skill evaluations (Figure 8) between RT-2 and two baselines.

(b) Ablations of RT-2-PaLI-X showcasing the impact of parameter count and training strategy on generalization.

Figure 6 | Quantitative performance of RT-2 across (6a) emergent skills and (6b) size and training ablations. Appendix Tables 5 and 6 detail the full numerical results.

❑ Chain-of-Thought

- Action 앞에 자연어 계획을 생성하는 구조 실험
- *Augment data* to include an additional “Plan” step and actual action tokens
  - Plan: describes the purpose of the action that the robot is about to take in natural language
  - VQA (Visual Q&A) 데이터의 *visual reasoning*과 manipulation 데이터의 *action generation*을 연결
  - Ex) “Instruction: I’m hungry. Plan: pick rxbar chocolate. Action: 1 128 124 136 121 158 111 255.”
- *Qualitatively* observe that RT-2 with CoT reasoning is able to answer more sophisticated commands
  - initial evidence that using LLMs or VLMs as planners can be combined with low-level policies in a single VLA model.

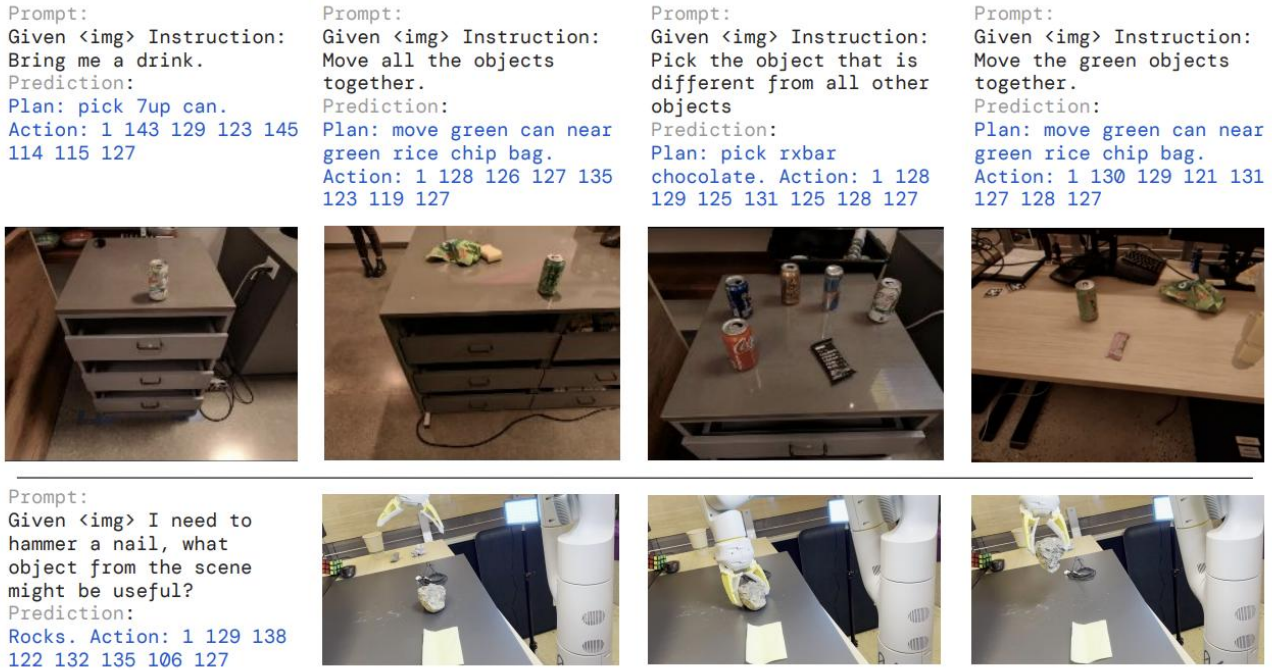


Figure 7 | Rollouts of RT-2 with chain-of-thought reasoning, where RT-2 generates both a plan and an action.

RT-2. Limitations

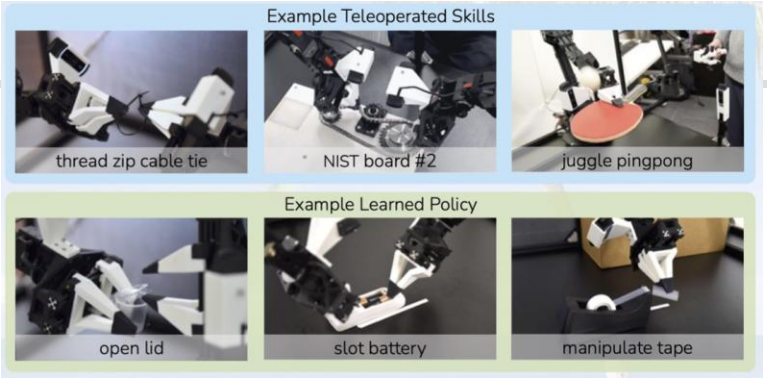
- Contributions
  - 로봇 정책의 학습 범위를 로봇 데이터 바깥으로 확장
    - Web-scale vision-language pre-trained knowledge (VLM)
    - 웹에서 학습된 시맨틱 지식을 함께 활용
  - 표현의 성격과 학습 방식 변화를 통한 일반화 성능 향상
    - 웹에서 학습한 Semantic 지식이 행동 선택에 반영
    - co-fine-tuning이 일반화 성능 우수
  - 복잡한 전용 구조 없이도 VLA 구현 가능성의 발견
    - 기존 VLM + Action Token 추가
- Limitations
  - Physical Skill 자체는 확장되지 않음
    - RT-2: 행동을 더 잘 "선택"하게 만들지만, 행동 자체를 새롭게 "생성"하지는 못함
    - Pre-training with Web scale data: 시맨틱·시각적 일반화
    - Robot Skill Demonstration data: 로봇이 수행할 수 있는 실제 동작
  - 계산 비용과 실시간성
    - 최대 55B Model을 Cloud 기반으로 사용 → 높은 계산 비용, 낮은 제어 주파수 (1-5Hz)
  - 모델 접근성과 생태계
    - Fine-tuning 가능한 대규모 VLM 필요

ACT

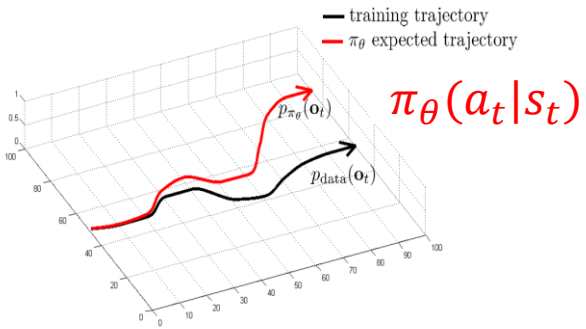
Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware

https://arxiv.org/pdf/2304.13705

ACT. summary

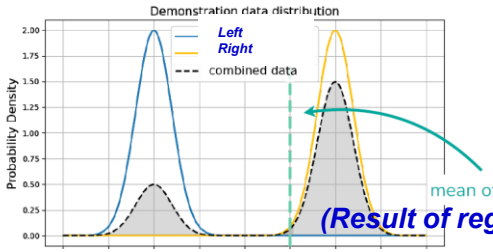


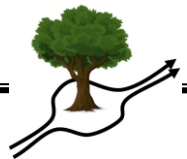
- ❑ 목적
  - Imitation Learning의 Compounding Error 절감
    - *정밀 조작*에서 발생하는 *누적 오차*를 줄이고자 한 모델
- ❑ 주요 제안 방법
  - Action Chunking
    - 한 동작 단위가 아니라, 짧은 *행동 묶음* 단위로 예측
  - Temporal Ensemble
    - 같은 시간의 여러 candidate action을 averaging
  - CVAE (z)
    - Continuous Joint Action의 *Multi-Modal Action Distribution* 고려
    - 동일 동작을 수행하는 다양한 demonstration data를 (action style) z로 표현하고, 이를 action 생성에 이용
  - Joint value를 직접 예측
    - RT-1, RT-2와 같이 action token (ex. Discrete bin index)를 예측한 후 de-tokenization 하는 대신 *(L1-regression)* 을 통해 각 관절 모터의 *(continuous) value* 를 직접 출력



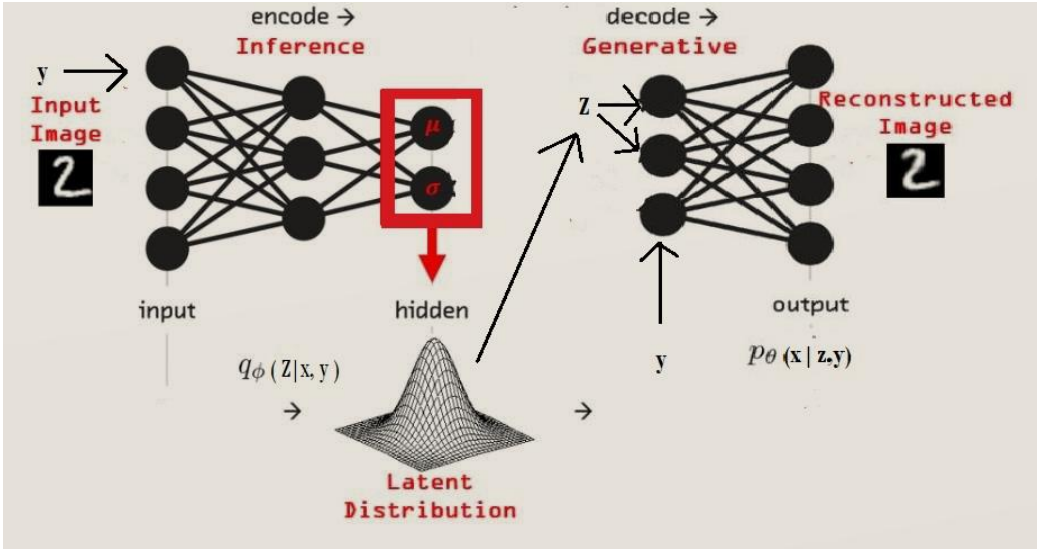
$\pi_\theta(a_t|s_t)$

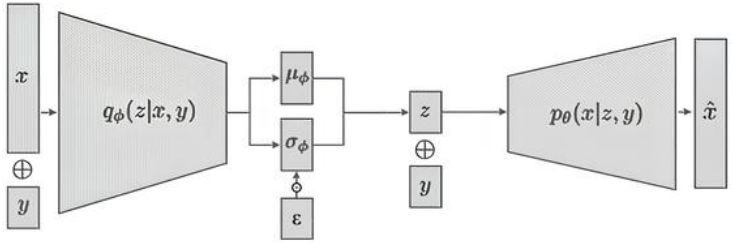


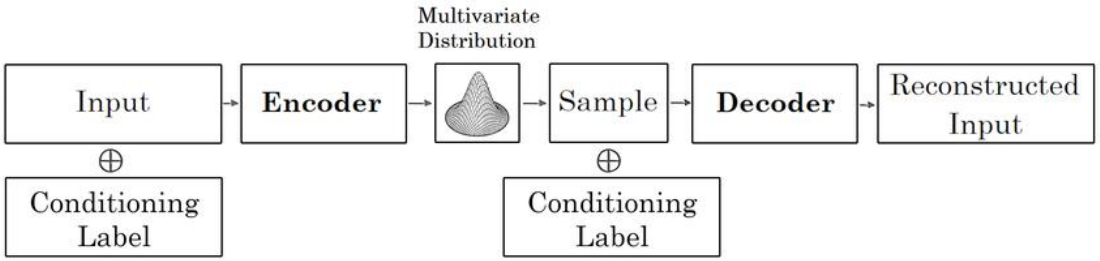




- CVAE (Conditional Variational Autoencoder)
  - VAE: encodes *input data* into a continuous, probabilistic *latent space* to reconstruct and create *new* data

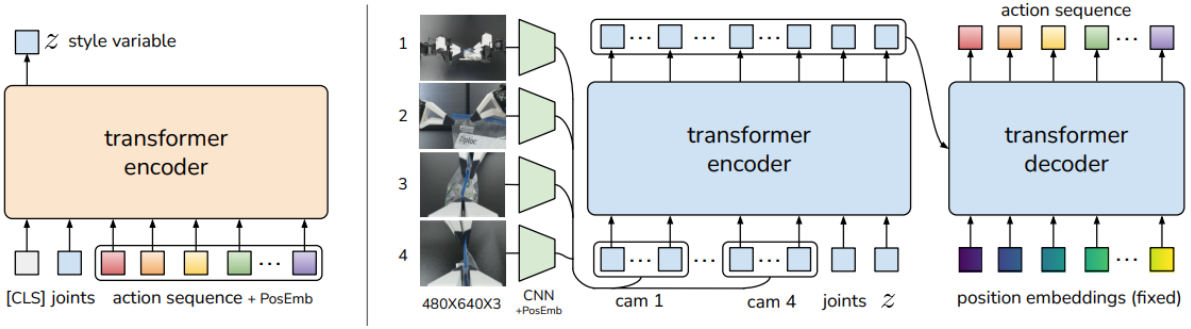






Act. 학습과 추론

- 학습 데이터
  - (현재 관측 정보, 사람이 실제 수행한 미래 action chunk)
    - 현재 관측 정보: *4개의 카메라* (위, 앞, 왼쪽 속목, 오른쪽 손목) 이미지, *현재 joint 값*



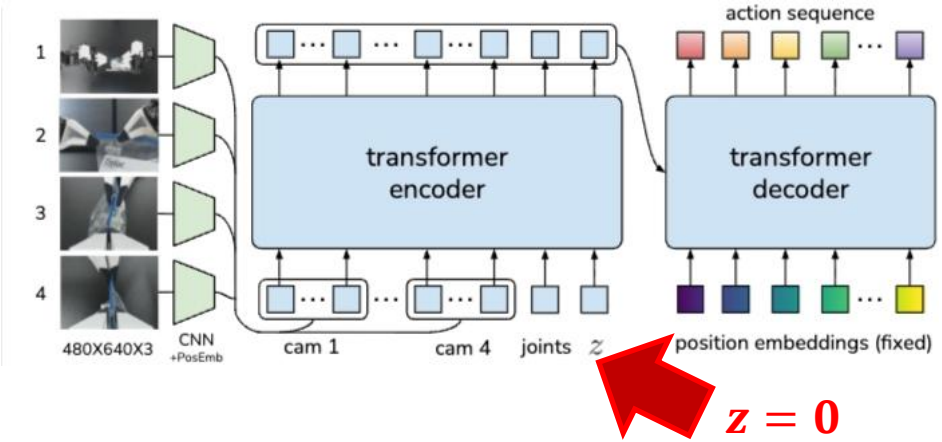
- 학습
  - Loss1 (재현): $\mathcal{L}_{reconst}$
    - 모델 출력 = demo data
    - L1 loss: $MSE(\hat{a}_{t:t+k}|o_t, z)$
  - Loss2(Regulation): $\mathcal{L}_{reg}$
    - (행동 스타일) z가 표준 정규분포와 유사
    - $D_{KL}(q_\phi(z|a_{t:t+k}, \bar{o}_t)||N(0,I))$

Algorithm 1 ACT Training

Given: Demo dataset $\mathcal{D}$, chunk size k, weight β.
Let a_t, o_t represent action and observation at timestep t, $\bar{o}_t$ represent o_t without image observations.
Initialize encoder $q_\phi(z|a_{t:t+k}, \bar{o}_t)$
Initialize decoder $\pi_\theta(\hat{a}_{t:t+k}|o_t, z)$
for iteration $n = 1, 2, \ldots$ **do**
  Sample o_t, $a_{t:t+k}$ from $\mathcal{D}$
  Sample z from $q_\phi(z|a_{t:t+k}, \bar{o}_t)$
  Predict $\hat{a}_{t:t+k}$ from $\pi_\theta(\hat{a}_{t:t+k}|o_t, z)$
  $\mathcal{L}_{reconst} = MSE(\hat{a}_{t:t+k}, a_{t:t+k})$
  $\mathcal{L}_{reg} = D_{KL}(q_\phi(z|a_{t:t+k}, \bar{o}_t) \parallel \mathcal{N}(0, I))$
  Update θ, ϕ with ADAM and $\mathcal{L} = \mathcal{L}_{reconst} + \beta\mathcal{L}_{reg}$

❑ 추론

- z 를 계산하기 위한 데이터가 없으므로 (← z를 계산하기 위해서는 action sequence가 필요하지만 추론할때 이 정보를 알 수 없음) z=0 (평균)을 사용



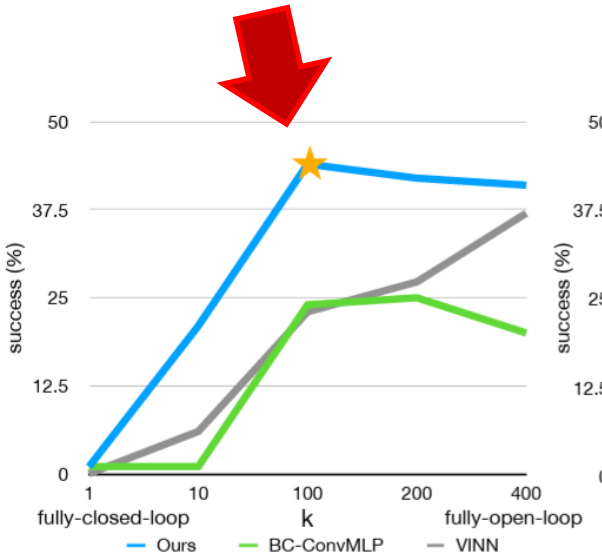
Algorithm 2 ACT Inference

1: Given: trained π_θ, episode length T, weight m.
2: Initialize FIFO buffers $\mathcal{B}[0 : T]$, where $\mathcal{B}[t]$ stores actions predicted *for* timestep t.
3: **for** timestep $t = 1, 2, ...T$ **do**
4: Predict $\hat{a}_{t:t+k}$ with $\pi_\theta(\hat{a}_{t:t+k}|o_t, z)$ where $z = 0$
5: Add $\hat{a}_{t:t+k}$ to buffers $\mathcal{B}[t : t + k]$ respectively
6: Obtain current step actions $A_t = \mathcal{B}[t]$
7: Apply $a_t = \sum_i w_i A_t[i] / \sum_i w_i$, with $w_i = \exp(-m * i)$

ACT. 성능 평가

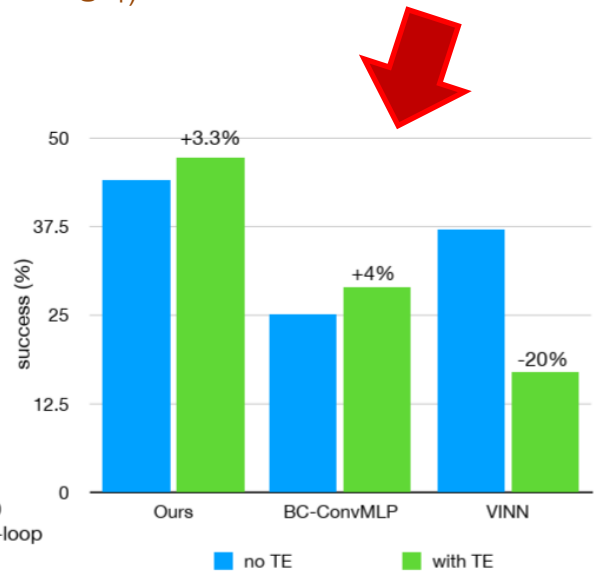
Action Chunking (k)

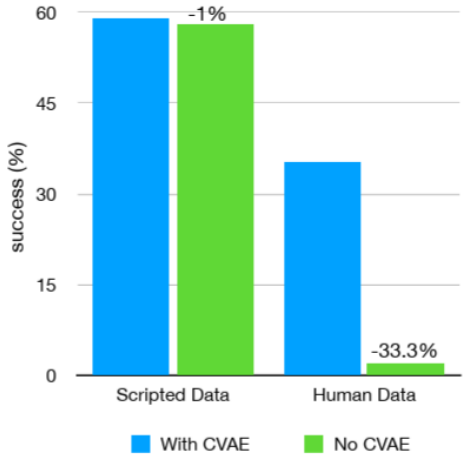
- Action chunk length (k)가 증가할 수록 성능 증가
- 길이거 일정 이상 길어지면 성능 하락 (prediction error 증가)



Temporal Ensemble의 영향

- ACT의 성능 3.3% 증가





Task	성공율(%)
Slide Ziploc	88
Slot Battery	96
Open Cup	84
Thread Velcro	20
Prep Tape	64
Put on Shoe	92

CVAE의 영향

- 스타일이 거의 같은 scripted data에서는 성능 변화 거의 없음
- 스타일 차이가 있는 human data의 경우 CVAE로 인해 성능 향상

OpenVLA

OpenVLA: An Open-Source Vision-Language-Action Model

https://arxiv.org/abs/2406.09246

https://openvla.github.io

OpenVLA

- 기존 문제점
  - 기존에 좋은 성능을 보여준 VLA 모델은 대체로 비공개
  - 모델 구조나 학습 절차, fine-tuning 방식이 충분히 공개되어 있지 않았음
- OpenVLA (7B 규모) 목표
  - 이미지와 언어를 이해하는 큰 모델 (VLM)을 바탕으로, 여러 로봇과 여러 조작(Manipulation) 작업에 사용할 수 있는 범용 로봇 정책을 만드는 것
    - VLM 기반 VLA (ex. RT-2)를 open-source로 구현
    - 새로운 환경에서도 fine-tuning할 수 있는 형태로 구현
- 학습 데이터
  - Open-X embodiment (https://robotics-transformer-x.github.io/) 데이터 중 970K robot episode data
    - 조작 (manipulation) Task data
    - 입출력 구조가 비교적 일관된 것
    - 최소 하나 이상의 외부 카메라가 있는 data
    - 출력에 로봇 팔의 end effector 위치 정보가 있는 data

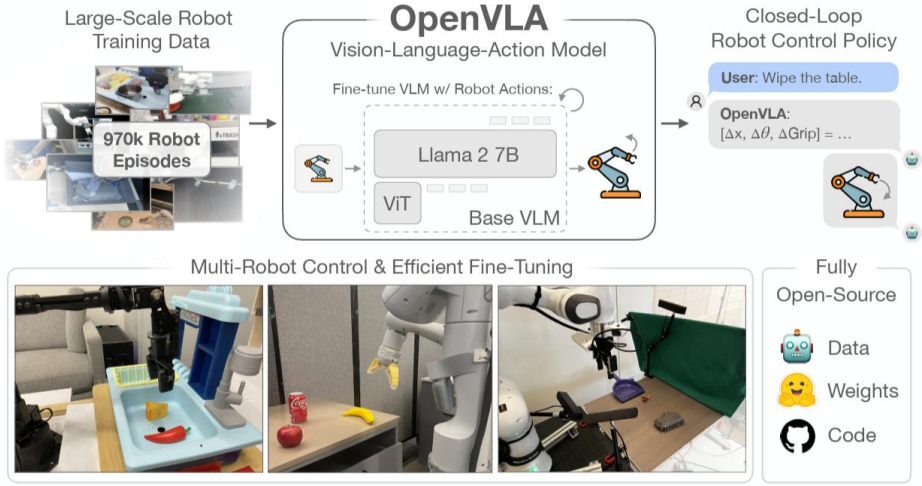
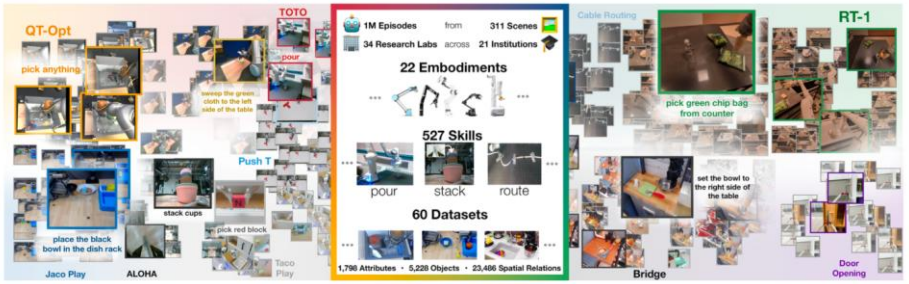


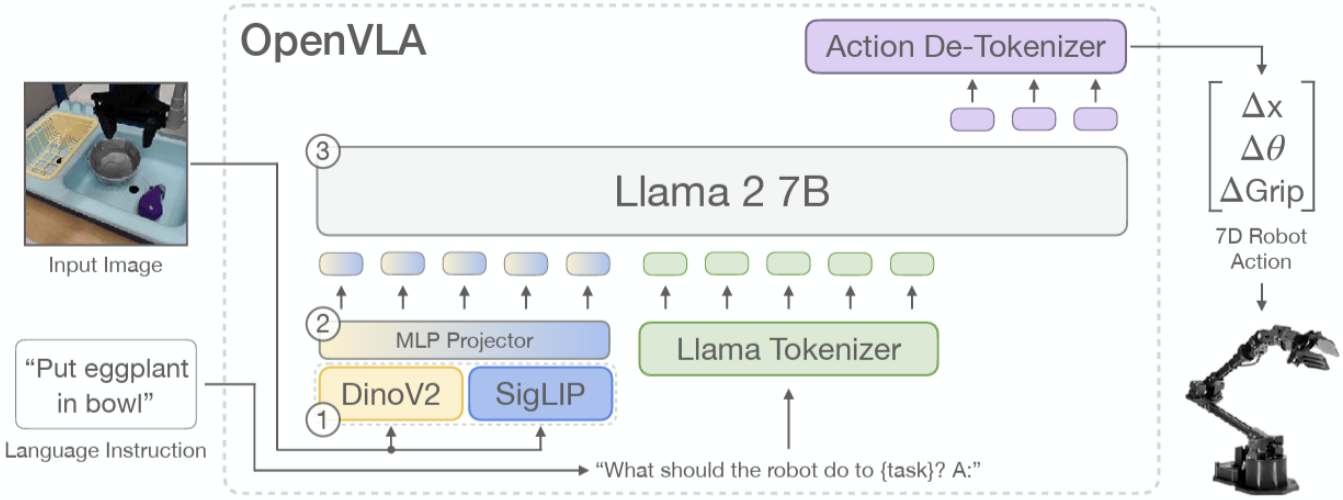
Figure 1: We present OpenVLA, a 7B-parameter open-source vision-language-action model (VLA), trained on 970k robot episodes from the Open X-Embodiment dataset [1]. OpenVLA sets a new state of the art for generalist robot manipulation policies. It supports controlling multiple robots out of the box and can be quickly adapted to new robot domains via parameter-efficient fine-tuning. The OpenVLA checkpoints and PyTorch training pipeline are fully open-source and models can be downloaded and fine-tuned from HuggingFace.



Open X-Embodiment Dataset: the largest open-source real robot dataset to date. It contains *1M+ real robot trajectories* spanning *22 robot embodiments*, from single robot arms to bi-manual robots and quadrupeds.

OpenVLA. 모델 구조

- 입력
  - 단일 이미지
  - 자연어 지시문
- 출력
  - 7차원 Cartesian Action
    - 로봇팔 끝부분의 위치/자세 변화, Gripper 상태: $(\Delta x, \Delta\theta, \Delta Grip)$
- Action Tokenization
  - 기존 VLM tokenizer에서 거의 사용하지 않는 256개 token 재활용
  - 각 action 값의 1%~99% 범위의 값들을 256개의 구간으로 나눔
    - cf. RT-2: 각 action의 최소~최대값 범위를 256개 구간으로 나눔
  - 이유
    - 로봇 데이터에 아주 드물게 큰 움직임이나 비정상적으로 튀는 action 값이 존재할 수 있음
    - 이런 값까지 모두 포함해서 최소값과 최대값을 잡으면, 전체 구간이 불필요하게 넓어짐



① Vision Encoder

- Concatenate DinoV2 and SigLIP
- SigLIP: Semantic feature (무엇이 있는가?)
- DinoV2: 저수준 공간정보 feature (어디에 어떻게 있는가?)
- Vision encoder fine-tuning이 pre-train보다 성능 향상
  - Pretrain된 일반 공간 정보에 로봇 제어를 위해 필요한 공간 정보 추가 학습

② Projector

- 2-layer MLP
- Vision 정보를을 language embedding space로 이동 시킴
  - 목적: LLM backbone 사용을 위해 (Llama tokenizer 출력 차원)

③ LLM Backbone

- Llama 2 7B

❑ Open-VLA

- Prismatic-7B VLM (vision encoder, projector, Llama 2 7B)을 robot action data로 VLM fine-tuning
- Action token sequence 출력

OpenVLA. 성능 평가

- Generalist Policy로 바로 사용 가능한가?
  - BridgeData V2의 WidowX와 RT-1/RT-2 평가에 쓰인 Google robot에서 평가
  - visual generalization
  - motion generalization
  - physical generalization
  - semantic generalization
  - language grounding

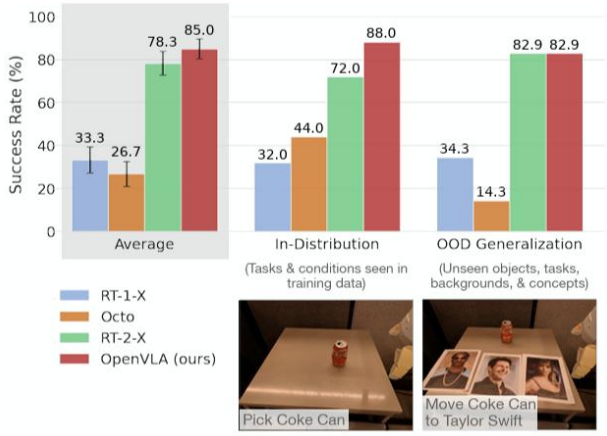


Figure 4: **Google robot evaluation results.** We evaluate generalist robot policies on in-distribution and out-of-distribution (OOD) tasks on the mobile manipulator used in RT-1 and RT-2 evaluations [2, 7]. We find that OpenVLA and RT-2-X attain comparable performance and significantly outperform RT-1-X and Octo overall. Average success rates ± StdErr are computed across 60 total rollouts per approach. See Table 6 for detailed results.

- 새로운 로봇과 새로운 작업에 잘 적응하는가?
  - Franka-Tabletop과 Franka-DROID 환경에서, task마다 10~150개의 demonstration만 사용해 OpenVLA fine-tuning
  - Narrow Single-Instruction Tasks
  - Diverse Multi-Instruction Tasks
    - Generalist OpenVLA 우수
  - 모든 task에서 최소 50% 이상의 성공율 (비교 대상 중 유일)

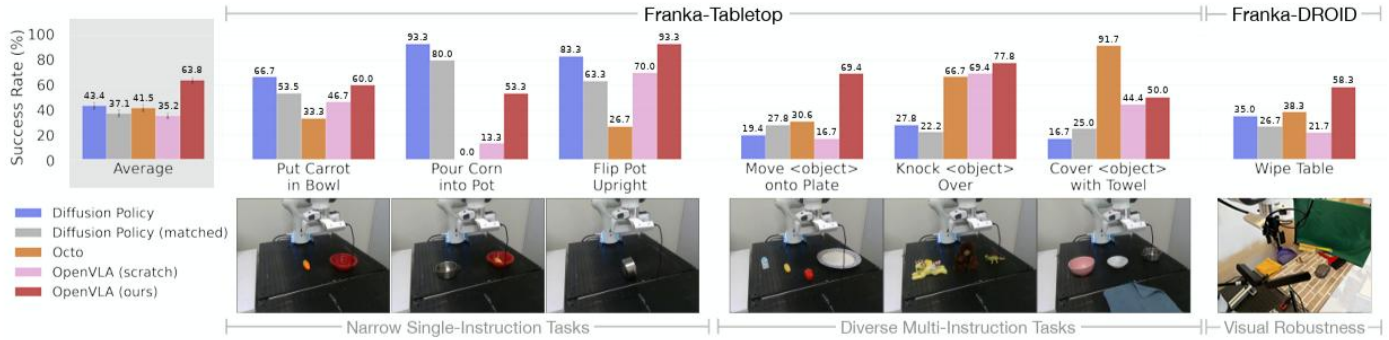


Figure 5: **Adapting to new robot setups.** We evaluate the state-of-the-art Diffusion Policy trained from scratch on seven Franka Emika Panda tasks (10–150 demonstrations each), as well as generalist robot policies Octo and OpenVLA fine-tuned on the same data. Diffusion Policy exhibits strong performance on narrow single-instruction tasks, while Octo and OpenVLA perform better on diverse fine-tuning tasks involving multiple instructions and distractor objects. Overall, OpenVLA achieves highest aggregate performance across both setups, suggesting that it is an effective default for learning a policy on a downstream task. Average success rates ± StdErr are computed across 129 rollouts per approach (99 for Franka-Tabletop tasks and 30 for Franka-DROID tasks). See Table 7 for detailed results.

❑ Parameter-Efficient Fine-Tuning

- LoRA: 단일 A100에서 10~15시간

Table 1: **Parameter-efficient fine-tuning evaluation.** LoRA fine-tuning achieves the best performance-compute trade-off, matching full fine-tuning performance while training only 1.4% of the model parameters. Mean success ± StdErr computed across 33 rollouts per approach on select Franka-Tabletop tasks (see Table 8 for details). *: Sharded across 2 GPUs with FSDP [77].

Strategy	Success Rate	Train Params ($\times 10^6$)	VRAM (batch 16)
Full FT	**69.7 ± 7.2 %**	7,188.1	163.3 GB*
Last layer only	30.3 ± 6.1 %	465.1	51.4 GB
Frozen vision	47.0 ± 6.9 %	6,760.4	156.2 GB*
Sandwich	62.1 ± 7.9 %	914.2	64.0 GB
LoRA, rank=32	**68.2 ± 7.5%**	**97.6**	**59.7 GB**
rank=64	**68.2 ± 7.8%**	195.2	60.5 GB

❑ Memory-Efficient Inference via Quantization

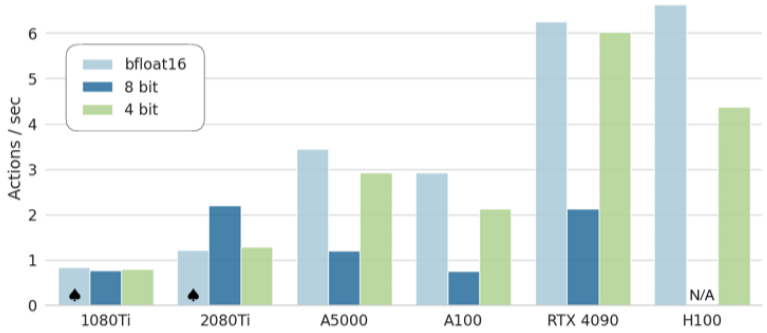


Figure 6: **OpenVLA inference speed for various GPUs.** Both bfloat16 and int4 quantization achieve high throughput, especially on GPUs with Ada Lovelace architecture (RTX 4090, H100). Further speed-ups are possible with modern LLM inference frameworks like TensorRT-LLM [89]. ♠: Model sharded across two GPUs to fit.

Precision	Bridge Success	VRAM
bfloat16	71.3 ± 4.8%	16.8 GB
int8	58.1 ± 5.1%	10.2 GB
int4	71.9 ± 4.7%	7.0 GB

Table 2: **Performance with quantized inference.** 4-bit quantization matches the performance of bfloat16 inference (our default approach) while reducing the GPU memory footprint by more than half. Mean success ± StdErr computed across 8 representative BridgeData V2 tasks [6] and 80 rollouts per approach (see Table 5 for details).

OpenVLA. 한계점

- 입력 다양성 부족
  - Single Image + Instruction
  - 멀티뷰 카메라, proprioceptive input, observation history 등 사용 안함
- 추론 속도
  - RTX 4090에서 약 6Hz로 동작
  - But, ALOHA 같은 시스템은 50Hz 수준의 고주파 제어를 요구
- 신뢰도
  - 이전 generalist policy보다 우수
  - 아직 매우 높은 신뢰도 수준은 아님
    - 대부분의 타스크에서 성공율 90% 미만

SmolVLA

SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics

https://arxiv.org/abs/2506.01844

SmolVLA

- 목적
  - VLA를 더 작고, 더 빠르고, 더 저렴하며, 더 많은 사람이 재현할 수 있는 형태로 개발
    - 로봇 모델은 논문 속 벤치마크에서만 잘하면 끝나는 것이 아님
    - 실제 로봇 위에서 돌아가야 하고, 반복 실험이 가능해야 하며, 가능하면 더 저렴한 하드웨어에서도 실행될 수 있어야 함
- SmolVLA
  - 작은 구조
    - VLM의 일부 layer만 활용하며, visual token 수를 줄입니다. 또한 Action Expert도 가볍게 설계
  - 공개 커뮤니티
    - 공개적으로 수집된 커뮤니티 데이터셋과 저가 로봇 플랫폼을 활용
  - 실제 로봇 배포를 고려한 추론 구조
    - 비동기 추론 구조를 통해 로봇이 action을 실행하는 동안 다음 action을 미리 계산하도록 설계
- 활용 예
  - SO-100/101, LeRobot, LeKiwi, etc. (https://huggingface.co/docs/lerobot/en/lekiwi)

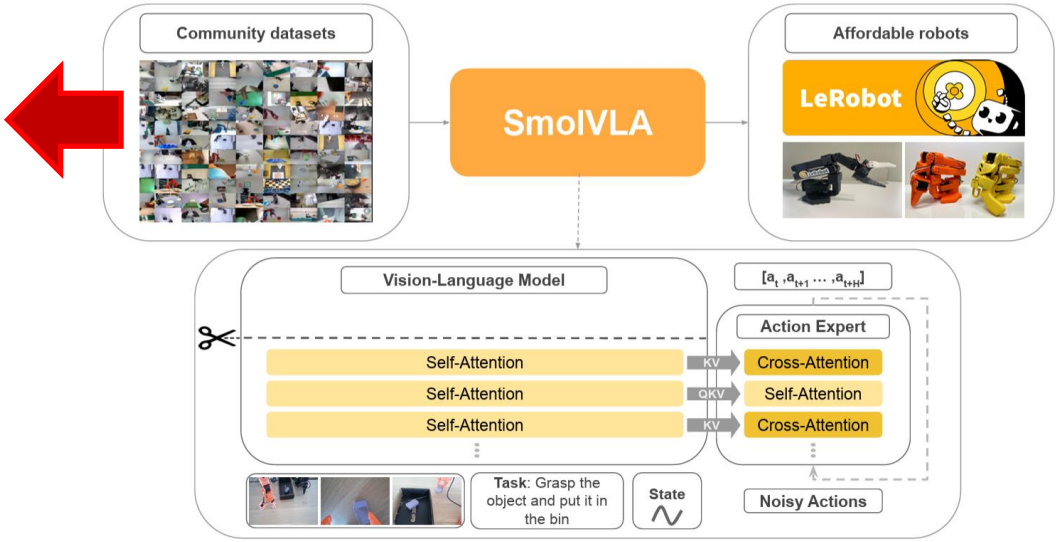
효율성	작은 모델, 적은 시각 토큰, layer skipping으로 학습·추론 비용 절감
접근성	단일 GPU 학습, 소비자급 GPU/CPU 배포, 저가 로봇 플랫폼 활용
재현성	공개 코드, pretrained model, training data, 커뮤니티 데이터셋 활용

SmolVLA. Model 구조

- 구성
  - Compact Pre-Trained VLM
  - Action Expert
- Design Choice
  - VLM의 마지막 layer 일부를 사용하지 않는 layer skipping (L=N/2)
  - 이미지 입력을 가볍게 만들기 위한 visual token reduction (64개)
  - 로봇 상태를 VLM 입력에 함께 넣는 state token 설계
  - cross-attention과 self-attention을 번갈아 쓰는 Action Expert
  - 로봇이 움직이는 동안 다음 action을 미리 계산하는 asynchronous inference

# datasets	# episodes	# frames
481	22.9K	10.6M

Table 1 | Community datasets statistics. At ~ 10M episodes, our pretraining set stands at least one order of magnitude smaller than other state-of-the-art. For completeness, we report the list of community-datasets used in Appendix A.1.



SmolVLA. 실험 결과

- Small *Size* (0.24B, 0.45B, 2.35B) v.s. OpenVLA(7B), π_0(3.3B)
  - Small size, but 성능 우수

Benchmark	Policy (# Params)	VLA Pt.	Success Rate (%) — Simulation				
LIBERO			**Spatial**	**Object**	**Goal**	**Long**	**Avg.**
	Diffusion Policy (Khazatsky et al., 2024)	No	78.3	92.5	68.3	50.5	72.4
	Octo (0.09B) (Team et al., 2024)	Yes	78.9	85.7	84.6	51.1	75.1
	OpenVLA (7B) (Kim et al., 2024)	Yes	84.7	88.4	79.2	53.7	76.5
	π_0 (Paligemma-3B)	No	87	63	89	48	71.8
	π_0 (3.3B)	Yes	90	86	95	73	86.0
	SmolVLA (0.24B)	No	87	93	88	63	82.75
	SmolVLA (0.45B)	No	90	96	92	71	87.3
	SmolVLA (2.25B)	No	93	94	91	77	**88.75**
Meta-World			**Easy**	**Medium**	**Hard**	**Very Hard**	**Avg.**
	Diffusion Policy (Chi et al., 2023)	No	23.1	10.7	1.9	6.1	10.5
	TinyVLA (Zhou et al., 2024)	No	77.6	21.5	11.4	15.8	31.6
	π_0 (3.5B-Paligemma)	No	80.4	40.9	36.7	44.0	50.5
	π_0 (3.5B)	Yes	71.8	48.2	41.7	30.0	47.9
	SmolVLA (0.24B)	No	86.43	46.36	35	60	56.95
	SmolVLA (0.45B)	No	82.5	41.8	45.0	60.0	57.3
	SmolVLA (2.25B)	No	87.14	51.82	70	64	**68.24**

	Success Rate (%) — Real World			
Policy	**Pick-Place**	**Stacking**	**Sorting**	**Avg.**
Single-task Training				
ACT	70	50	25	48.3
Multi-task Training				
π_0 (3.5B)	100	40	45	61.7
SmolVLA (0.45B)	75	90	70	78.3

Table 3 | Real-world benchmarks (SO100). Success rate (%) across three tasks using policies trained in multi-task and single-task settings.

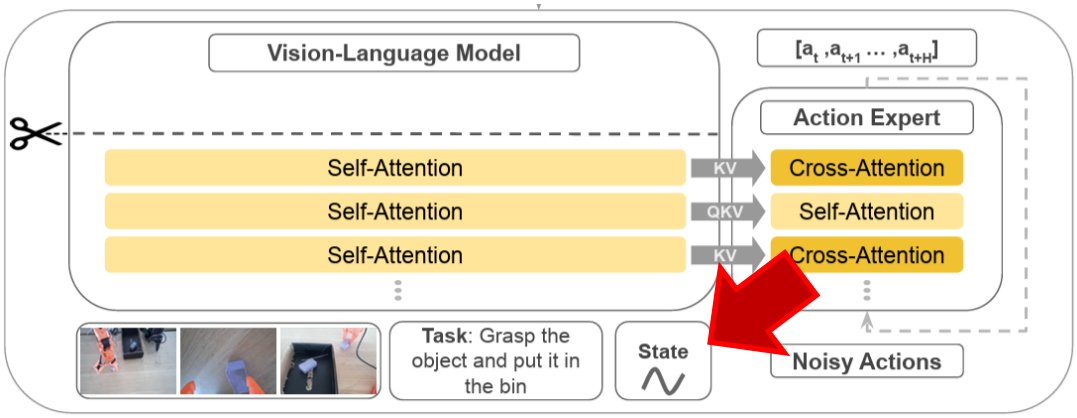
	Success Rate (%) — Real World	
Policy	**In Distribution**	**Out of Distribution**
Single-task Training		
ACT	70	40
SmolVLA (0.45B)	90	50

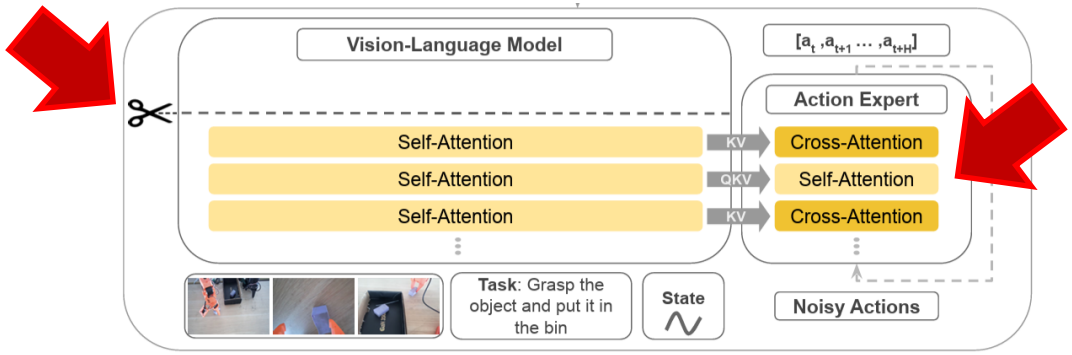
Table 4 | Real-world benchmark (SO101). Success rate (%) for the Pick-Place-Lego task using policies trained in single-task setting.

- 로봇 상태를 VLM 입력에 함께 넣는 *state token* 설계
  - Prefix: VLM의 vision+language 입력 다음으로 로봇 상태 (state) 입력
  - Suffix: Action Expert의 첫번째 입력으로 로봇 상태 입력
  - Prefix 방식이 Suffix 방식 보다 우수

States	Attention	Success Rate (%) — LIBERO				
		S	O	G	10	Avg
Prefix	CA	89	94	85	53	80.3
Suffix	CA	86	82	78	47	73.3
Prefix	SA	62	74	57	20	53.3
Suffix	SA	80	92	80	47	74.8

Table 11 | States as prefix vs. suffix. Feeding states to the VLM (prefix) leads to better performance than feeding them to the expert (suffix).





❑ *Layer Skipping*

Attention mechanism	Success Rate (%) — LIBERO				
	S	O	G	10	Avg
CA	87	92	83	54	79.0
SA	80	94	84	40	74.5
CA+SA (ours)	86	99	90	67	85.5

Table 6 | Cross vs self-attention. Interleaved cross (CA) and self-attention (SA) yield the best results.

❑ *Attention Mechanism (CA, SA)*

N	Success Rate (%) — LIBERO				
	S	O	G	10	Avg
8	77	88	86	49	75.0
16	88	91	91	44	78.5
24	86	97	86	49	79.5
32	89	94	85	53	80.3
Skip %2	84	90	83	45	75.5
VLM-256M	86	83	75	59	75.8

Table 8 | Skipping VLM layers. Skipping layers from a large VLM (here 500M parameters) yields better results than downsizing the VLM. Skipping every second layer is a competitive baseline.

Attention mask	Success Rate (%) — LIBERO				
	S	O	G	10	Avg
Bidir	79	86	82	23	67.5
Causal	80	94	84	40	74.5

Table 7 | Bidirectional vs causal attention. Preventing future action leakage improves performance.

❑ Action Chunk Size

Chunk Size	Success Rate (%) — LIBERO				
	S	O	G	10	Avg
1	45	77	54	24	50.0
10	90	94	94	58	84.0
30	85	94	87	48	78.5
50	89	94	85	53	80.3
100	83	88	85	42	74.5

Table 12 | Action chunk size. A chunk size between 10 and 50 strikes a good balance between action prediction frequency and performance.

❑ Action Execution Steps

Action Steps	Success Rate (%) — LIBERO				
	S	O	G	10	Avg
1	89	94	85	53	80.3
10	89	94	91	57	82.8
30	76	91	74	42	70.8
50	54	70	58	25	51.8

Table 13 | Action execution steps. Sampling new observations more frequently (e.g., every 1 or 10 steps) significantly improves performance.

❑ Training Objective

Training objective	Success Rate (%) — LIBERO				
	S	O	G	10	Avg
Flow matching	89	94	85	53	80.25
Regression	92	85	86	38	75.25

Table 10 | Training objective. We compare our training objective based on Flow matching to regression with L1 loss.

❑ *Asynchronous Inference*

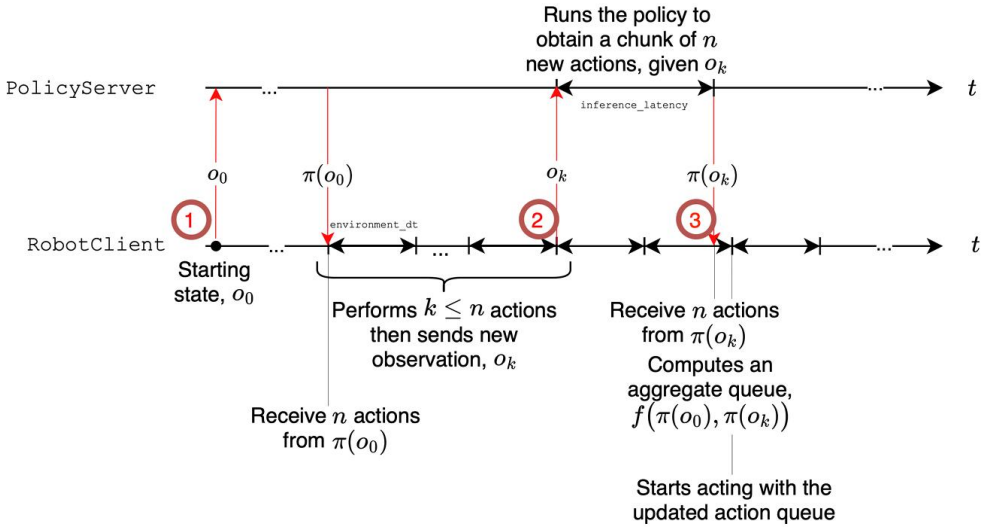


Figure 2 | Asynchronous inference. Illustration of the asynchronous inference stack. Note that the policy can be run on a remote server, possibly with GPUs.

Inference	Success Rate (%) — Real World			
	Pick-Place	Stacking	Sorting	Avg
Sync	75	90	70	78.3
Async	80	90	50	73.3

(a) | Performance (success rates).

Inference	Time (s) — Real World		
	Total	Avg	Std
Sync	137.5	13.75	2.42
Async	97.0	9.70	2.95

(b) | Task completion time.

Inference	# of Cubes — Real World		
	Total	Avg	Std
Sync	9	1.8	0.45
Async	19	3.8	1.3

(c) | Performance in fixed time.

Figure 5 | Comparison between synchronous (Sync) and asynchronous (Async) inference. We evaluate SmolVLA on three real-world tasks under both inference modes. Asynchronous inference achieves similar success rates (left) but is significantly faster (middle) and complete more tasks (right) in fixed-time settings. Hyperparameters have been optimized for Pick-Place and reused across tasks.

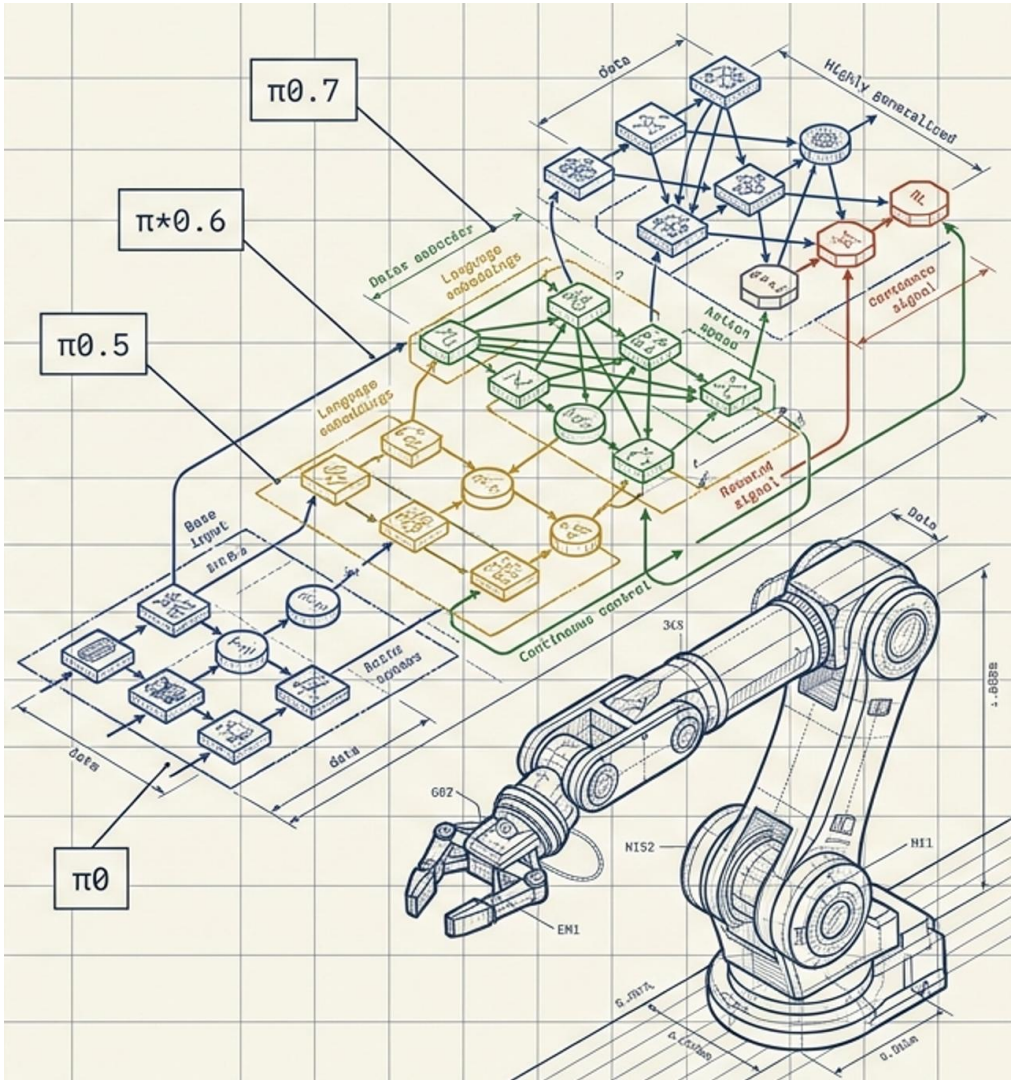
SmolVLA. 한계점

- Dataset diversity and cross-embodiment training
  - Datasets collected from a single robot type (SO100)
- Dataset size and scalability
  - Trained with approx. 23K trajectories (cf. OpenVLA: 1M trajectories)
- Model size (under 0.5B) and hardware efficiency
  - exploring ways to scale these architectures further without sacrificing speed or accessibility is an important direction for future research.
- Choice of VLM backbone
  - rely on an off-the-shelf VLM backbone pre-trained mainly on document reading and OCR tasks (https://arxiv.org/abs/2504.05299)
  - it is not yet clear whether these VLMs are optimal for real-world robotic interaction scenarios
- Multi-modal and robotics data joint training
  - to improve generalization and instruction-following abilities.
- Task complexity and longer horizon
  - Incorporating hierarchical policies or multi-level planning mechanisms may help to address this complexity.
- Learning paradigms: Imitation vs. Reinforcement Learning
  - RL could offer significant performance benefits and more dexterous policy adaptation (,especially for handling complex or long–horizon tasks)

π Series Evolution

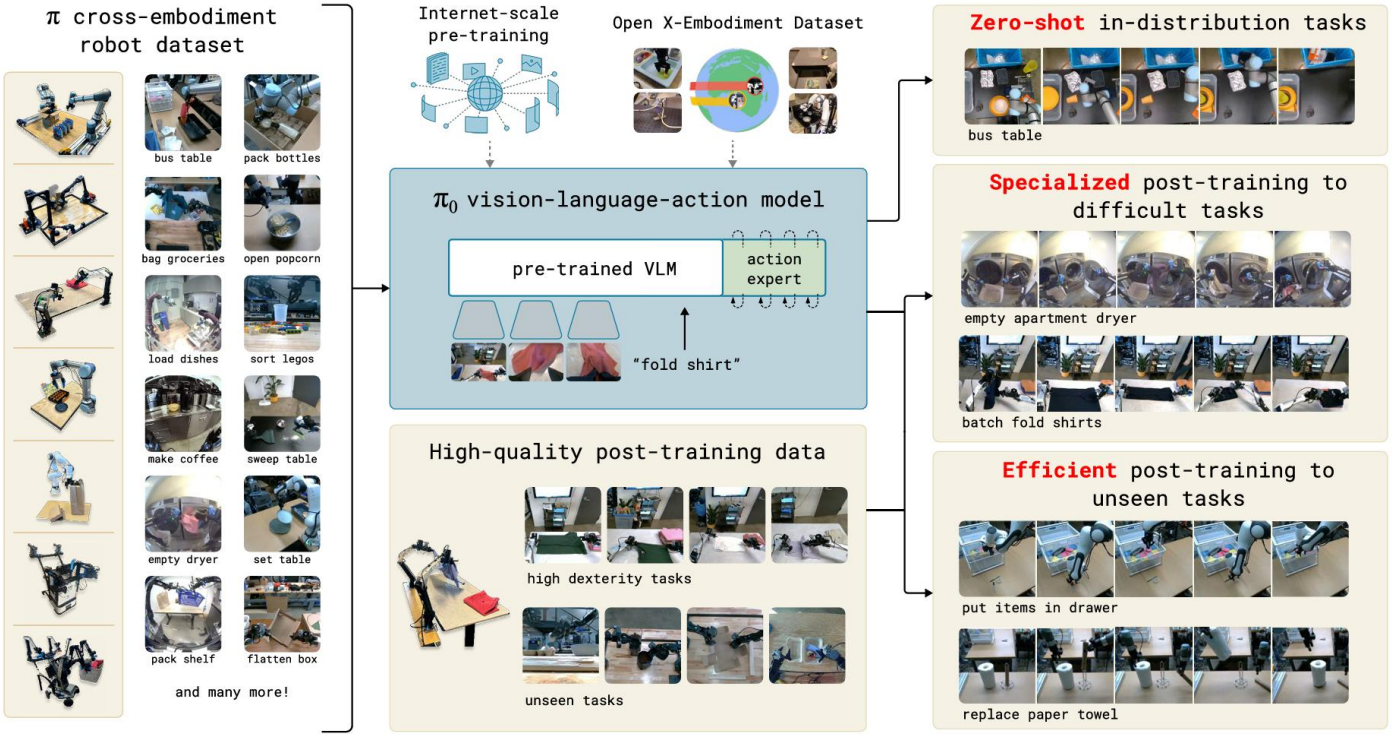
Physical Intelligence

https://www.pi.website/



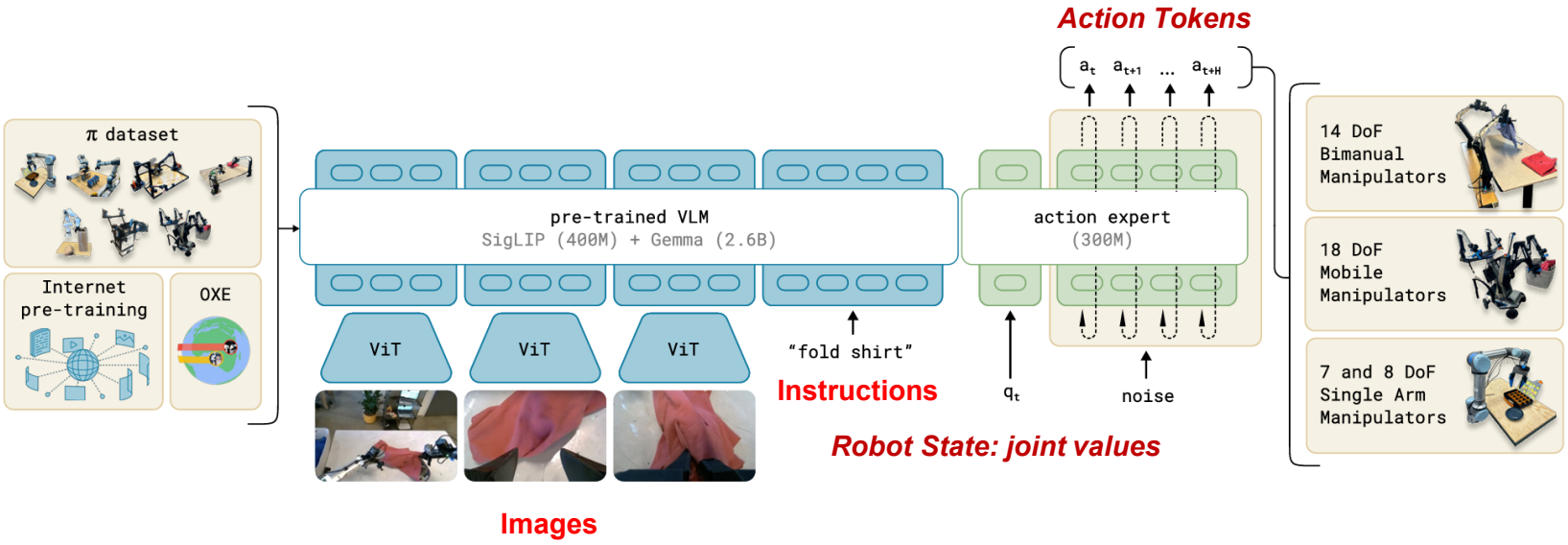
π_0

- 기존 VLA (RT-2, OpenVLA)
  - Contribution: 인터넷의 방대한 시각-언어 지식을 로봇에 이식
  - 단점
    - 물리적 움직임의 정교함
    - 실시간 대응 능력 (Control Frequency $\leq 6Hz$)
- π_0: VLM (PaliGemma) + Action Expert
  - Flow Matching을 이용한 Action Chunk Generation



π_0: Framework Overview

- VLM backbone weights are initialized from PaliGemma
  - Providing representations learned from large-scale Internet pre-training
- Start with a pre-training mixture to train VLM and Action Expert
  - π dataset (dexterous manipulation)
  - Open-source data (OpenX-embodiment, Bridge v2, DROID)
- Resulting π_0 model can
  - control multiple robot embodiments with differing action spaces to accomplish a wide variety of tasks



π_0. 학습 데이터

- Pre-Training Dataset
  - Open-Source dataset
    - robots and tasks in these datasets typically have one or two cameras and use low frequency control, between 2 and 10 Hz
    - However, these datasets cover a wide range of objects and environments.
  - Their own dataset (903M timesteps)
    - has 68 tasks, where each task is composed of *complex* behaviors — e.g., the "bussing" task involves putting a *wide range of different objects (*dishes, cups, utensils, and a wide array of trash items)
    - Note that this definition of task is significantly different from prior work, which typically uses any combination of noun and verb (e.g., "pick up the cup" vs. "pick up the plate") to constitute a distinct task.
    - Therefore, the actual *range of behaviors* in our dataset is *significantly broader* than this number of "tasks" would imply
- Post-Training Dataset
  - 5hours ~ 100hours of data, depending on task complexity
  - To fine-tune our model with a smaller task-specific dataset to specialize it to particular downstream applications.

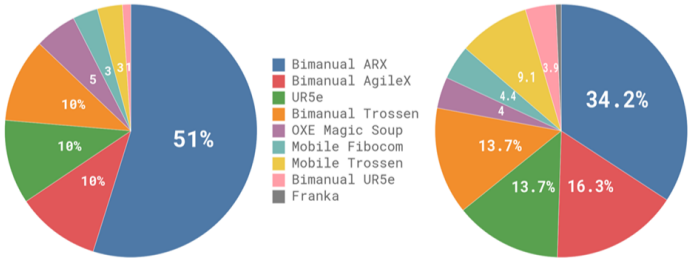


Fig. 4: **Overview of our dataset**: The pre-training mixture consists of a subset of OXE [10] and the π dataset. We use a subset of OXE, which we refer to as OXE Magic Soup [24]. The right figure illustrates the weight of the different datasets in the pre-training mixture. The left figure illustrates their relative sizes as measured by the number of steps.

Data Curation

- 각각의 task-robot combination에 해당하는 data 양 (# of timesteps)에 따라 가중치 조절하여 양이 다른 dataset들을 balance
- $n^{0.43}$ (n: task-robot combination dataset의 양, in terms of # of timesteps)



π_0. 성능 평가

❑ *Out-of-Box Evaluation (Zero-Shot)*

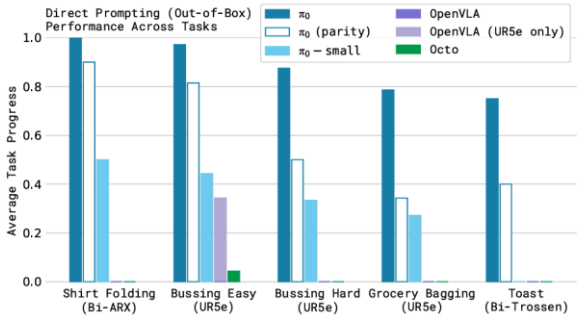


Fig. 7: **Out-of-box evaluation results**: We evaluate π_0 trained for the full 700k steps, a version trained for 160k steps that matches the number of updates for baseline models, π_0-small, and three baselines: OpenVLA and Octo trained on all of our data, and OpenVLA trained only on the UR5e tasks (which we found to work better on UR5e tasks). Across all tasks and all comparisons, even the "parity" version of our model outperforms all baselines, and the full version of our model achieves the best results by a large margin.

❑ *Language Following*

- Flat: directly command (ex. "bus the table")
- Human, HL: semantic reasoning and high level strategies (i.e., intermediate subtasks)
  - Ex) "pick up the napkin"

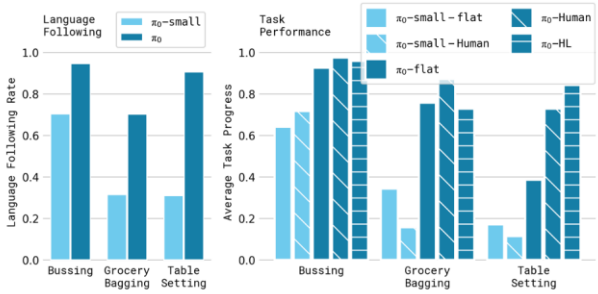


Fig. 9: **Language evaluation.** We compare "flat" versions of our policies, `–flat`, which receive only the overall task command (e.g., "bag the groceries") with a method that receives intermediate commands from a human expert, `–human`, or a high-level VLM policy, `–HL`. We also compare our model to a small non-VLM variant under the "expert" condition, π_0 and π_0-small, in terms of language following accuracy. The results show a significant improvement with π_0 from intermediate language commands provided by a human expert and to a lesser degree by an autonomous high-level policy. Notably, due to π_0-small's limited language following ability, overall it does not gain with the addition of a high-level expert.

❑ *Post-Training Results on Complex Tasks*

- Fine-tuned: after pre-training and fine-tuning
- Scratch: training on the fine-tuning data without any pre-training (task specific)

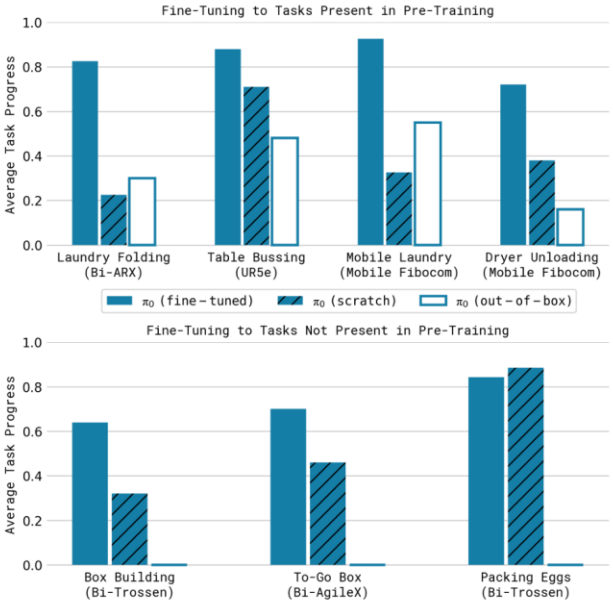


Fig. 13: **Post-training results on complex tasks** in terms of average scores over 10 trials. The full pre-trained π_0 model attains more than 50% of the maximum score across all of the tasks, and typically outperforms the ablations, with especially significant improvements on the hardest tasks.

❑ *Fine-Tuning*

- Fine-tuned: after pre-training and fine-tuning
- Scratch: training on the fine-tuning data without any pre-training (task specific)

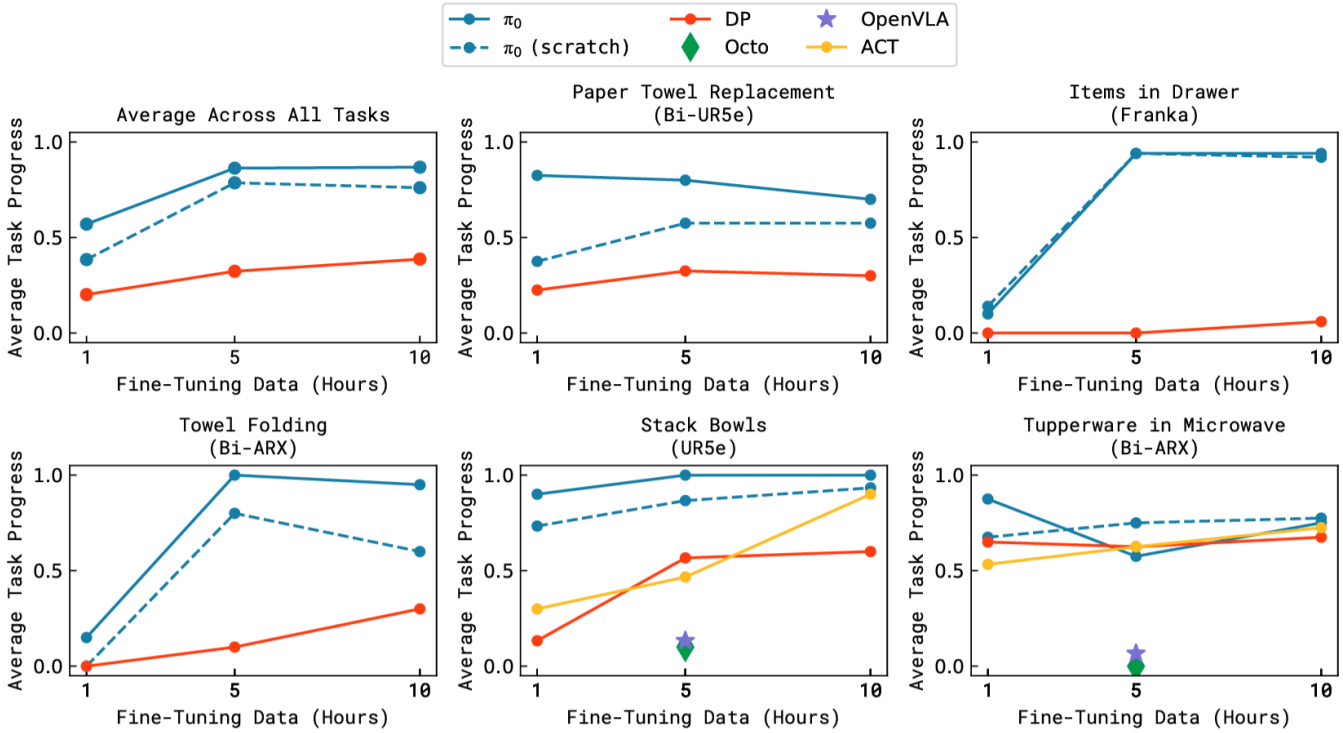


Fig. 11: **Fine-tuning with varying amounts of data.** π_0 can learn some easier tasks even with smaller amounts of data, and the pre-trained model often attains a larger improvement over the model trained from scratch.

$\pi_{0.5}$ for Open-World Generalization (https://www.pi.website/blog/pi05)

- π_0의 문제점
  - 정해진 환경 내에서 고난도 동작 성능 우수
  - 새로운 환경에 배치될 경우 일반화 성능 급격히 감소
- $\pi_{0.5}$의 목표
  - 새로운 환경에서 고차원적 지시를 이해 (subtask로 분해하여 단계별 처리)하고 처음보는 가구 및 객체들과 상호 작용하며 10~15분의 ***long task*** 수행
  - Main Contribution
    - 고수준 의미 판단 ***(high-level semantics)***과 저수준 정밀 제어 ***(low-level control)***를 하나의 모델 안에서 연결
  - https://www.pi.website/blog/pi05

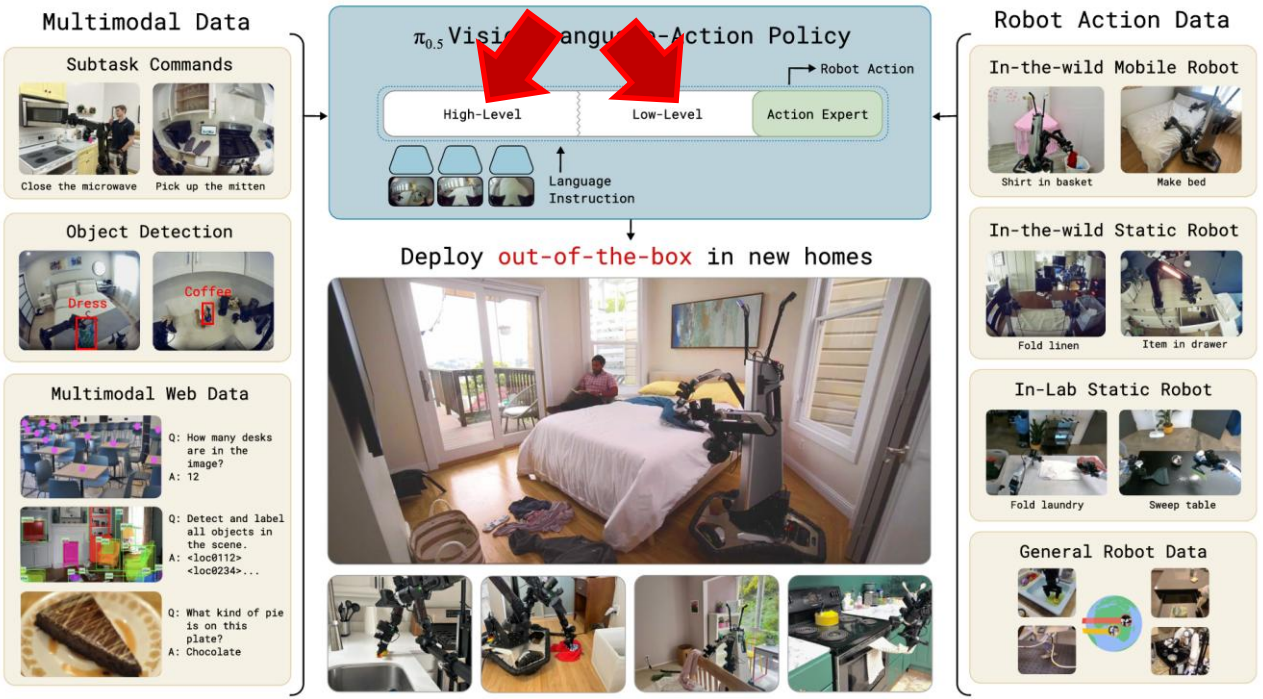
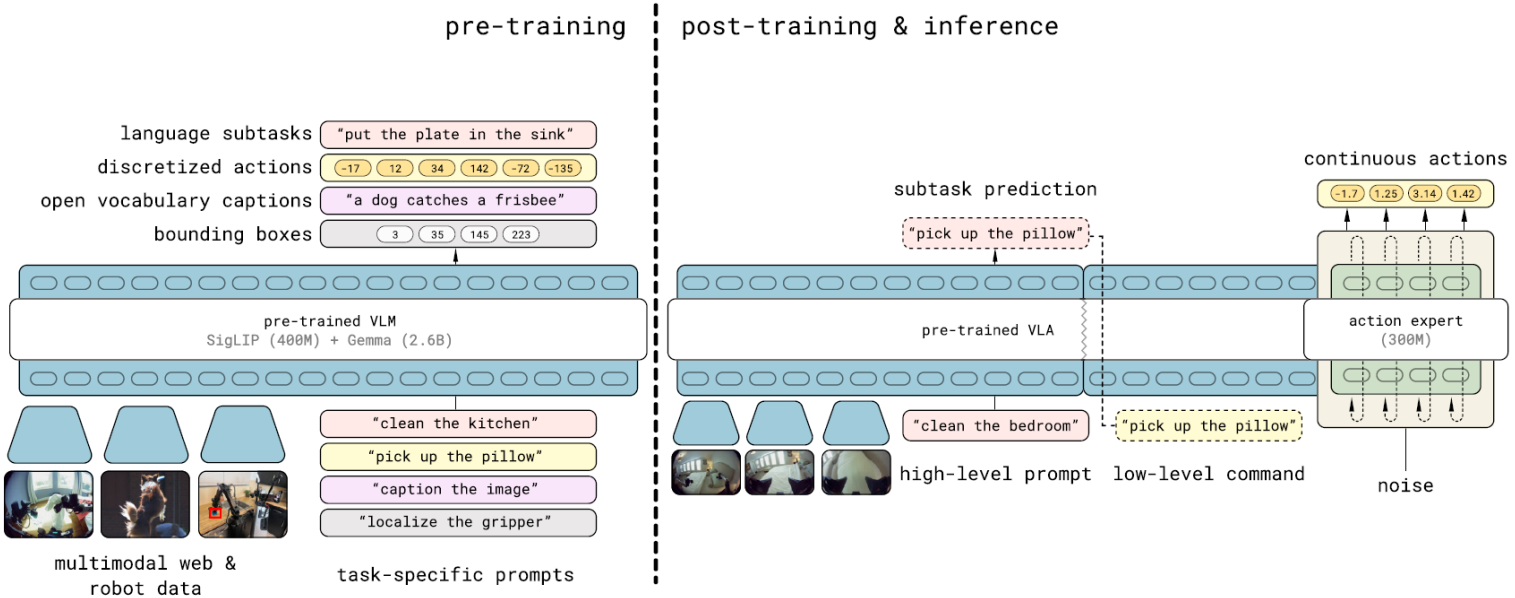


Fig. 1: The $\pi_{0.5}$ model transfers knowledge from a heterogeneous range of data sources, including other robots, high-level subtask prediction, verbal instructions, and data from the web, in order to enable broad generalization across environments and objects. $\pi_{0.5}$ can control a mobile manipulator to clean kitchens and bedrooms in new homes that were not present in the training data, performing complex multi-stage behaviors with durations of 10 to 15 minutes.

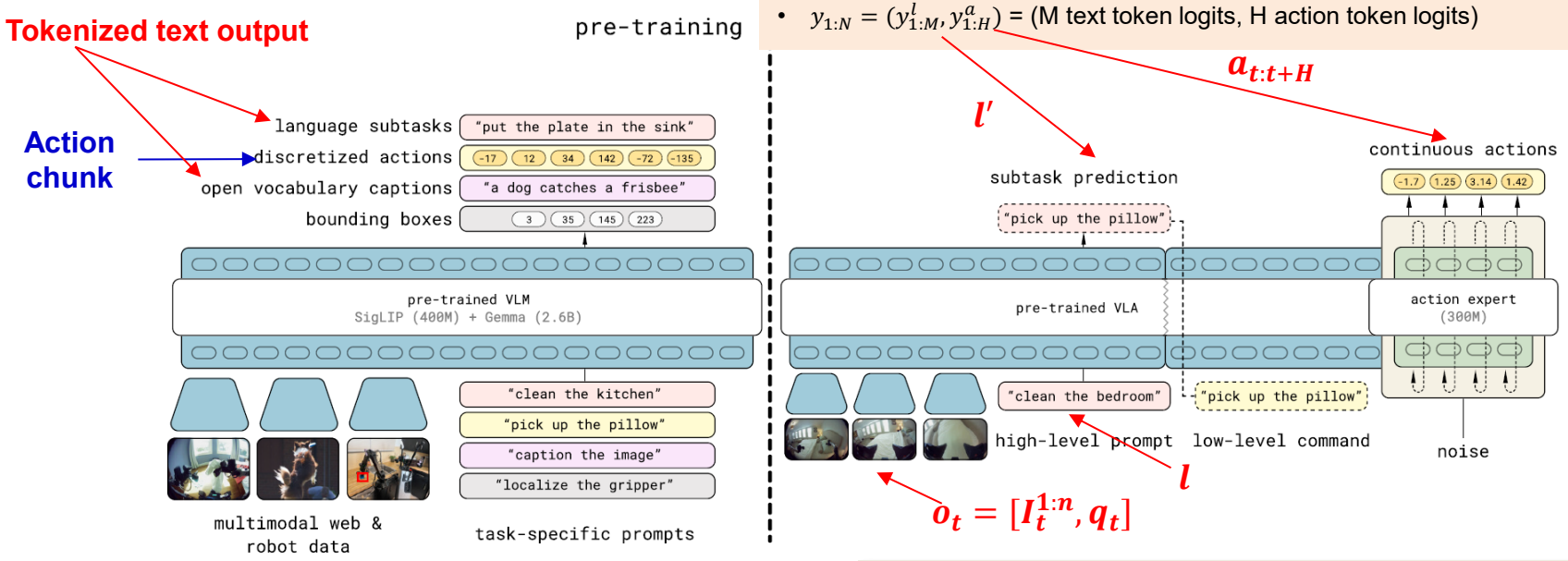
$\pi_{0.5}$ Model Overview

- Pre-Training: 대규모 학습에 필요한 빠른 연산을 위해 discrete token 방식 사용
  - combines *all of the different data* sources (*robot, non-robot*) to produce an *initial VLA with discrete tokens*
    - uses data from diverse robotic platforms, high-level semantic action prediction, and data from the web.
- Post-Training: 정밀도가 요구되는 실제 로봇 제어를 위해 continuous action 학습
  - specializes the model for low-level and high-level inferences for mobile manipulation,
    - leveraging the most task-relevant data, including verbal instructions from human supervisors
  - uses flow matching to represent the action distribution,
    - enabling efficient real-time inference and the ability to represent fine-grained continuous action sequences.
- Inference
  - first infers a high-level *subtask*, and then predicts the *actions based on this subtask*



Model outputs: $y_{1:N} = f(x_{1:N}, A(x_{1:N}), \rho(x_{1:N}))$

- $x_{1:N}$: input tokens ($x_i \in$ {text, image, intermediate denoising action})
- $A(x_{1:N}) \in [0,1]^{N \times N}$: Attention matrix (=1 when a token attend another token)
- $\rho(x_{1:N})$: token type of each input x_i (determining different expert and weights: MoE)
- $y_{1:N} = (y^l_{1:M}, y^a_{1:H})$ = (M text token logits, H action token logits)



- ❑ $\pi_{0.5}$ (i.e., one model) can flexibly represent *both*
  - Action chunk distribution and
  - Tokenized test output
    - Question-answering
    - High-level subtask predictions
- ❑ Training Loss
  - Cross Entropy (text) + Regression (action)
  - $H(x_{1:M}, y^l_{1:M}) + \alpha \| w - a_{t:t+H} - y^a_{1:H} \|^2$

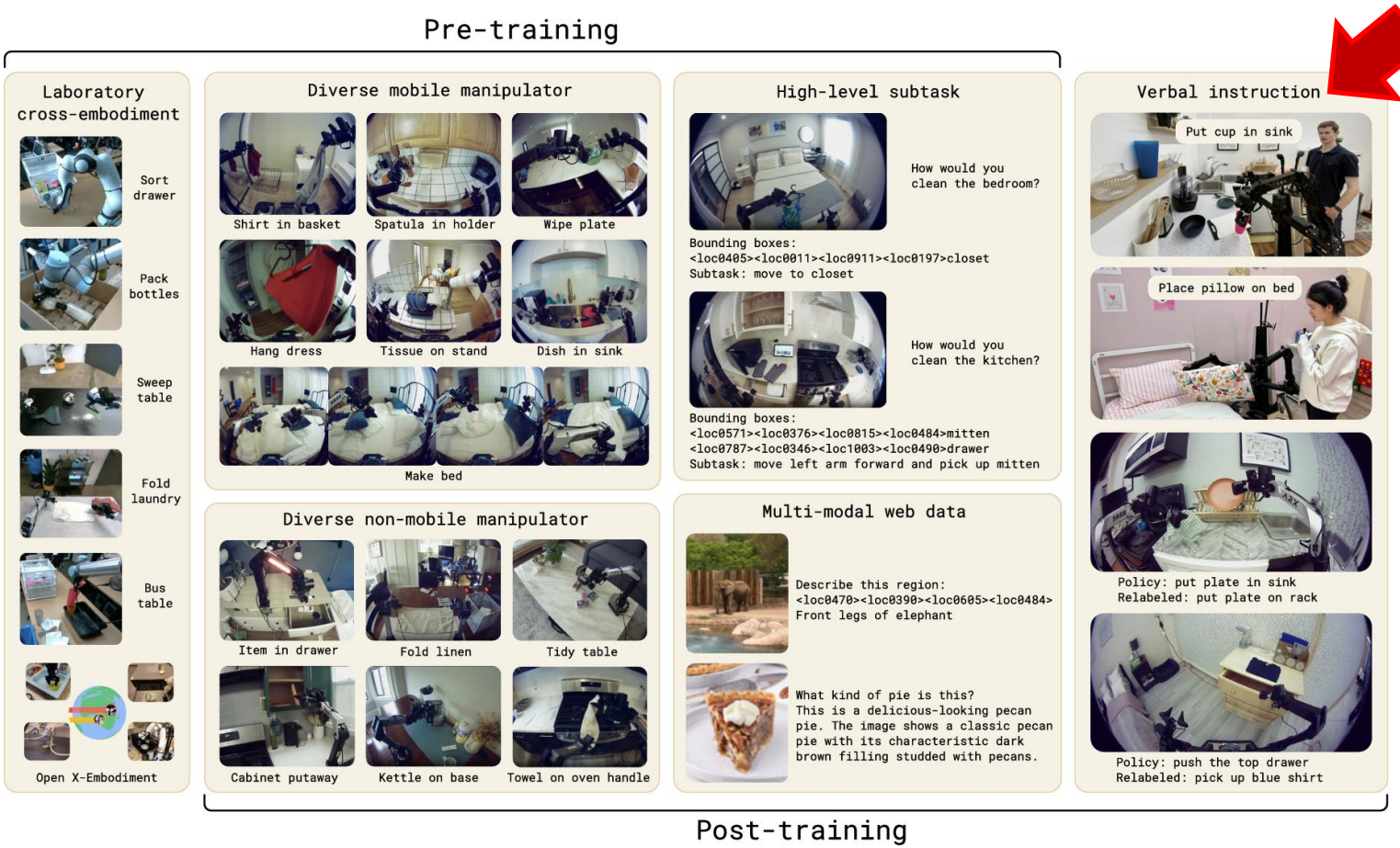
- ❑ Model $\pi_{0.5}$ captures distribution

$$\pi_\theta(a_{t:t+H}, l' | o_t, l) = \pi_\theta(a_{t:t+H} | o_t, l') \pi_\theta(l' | o_t, l)$$

  - o_t: images and robot state
  - l: overall task prompt
  - l': model's textual (subtask or Q&A)
  - $a_{t:t+H}$: predicted action chunk

← the action distribution does not depend on l, only on l'

$\pi_{0.5}$: Pre-Training and Post-Training Data



❑ Pre-Training

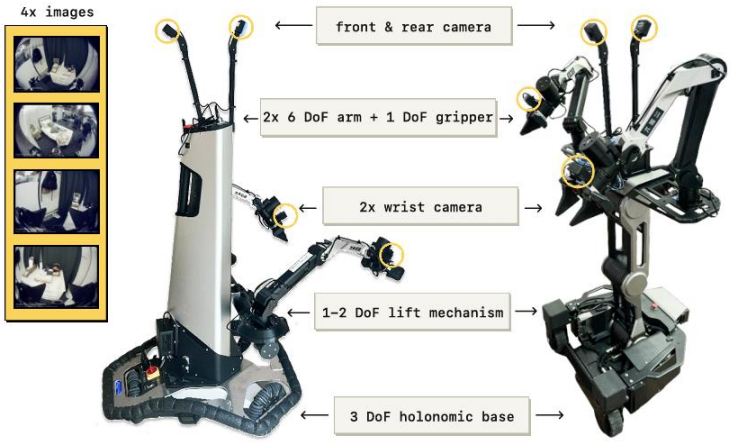
- 로봇 데이터: 모바일 매니퓰레이터(MM), 다양한 환경의 비모바일 로봇(ME), 실험실 내 교차 형태 로봇(CE) 데이터를 이산 토큰 방식(FAST tokenizer 활용)으로 학습
- 비로봇 데이터: 웹 이미지 캡셔닝, QA, object localization(객체 위치 추정) 데이터와 더불어 작업을 세분화하여 예측하는 CoT(Chain-of-Thought) 형태의 하위 작업 예측 데이터를 활용

❑ Post-Training: 실제 가정환경에서 use case에 맞게

- Action Expert 최적화: Flow Matching 기법을 통해 연속적인 Action Chunk를 예측하도록 Action Expert를 집중 학습합니다.
- Self-Prompting: 하위 작업 데이터를 통해 모델이 스스로 다음 단계를 먼저 예측(subtask prediction)하고 움직이도록 유도하여 복잡한 멀티스테이지 작업 성공률을 높임.

$\pi_{0.5}$: Robot System Detail

- Mobile Manipulators
  - 4 RGB cameras (front, rear, left wrist, right wrist)
  - Two 6 DoF arms with parallel jaw grippers
  - a wheeled holonomic base
  - a torso lift mechanism
- Total dimensionality of state and action space
  - 18-19 DoFs

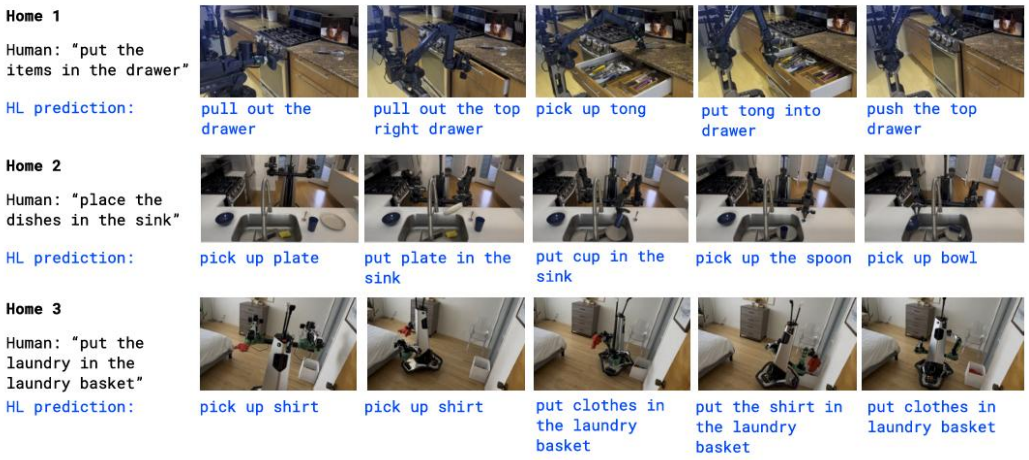


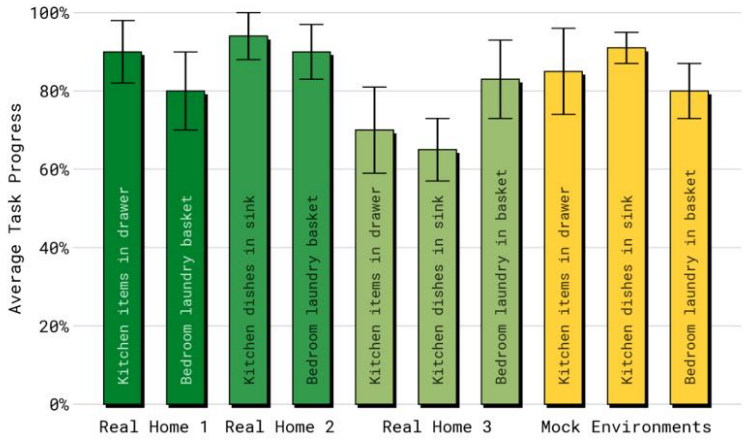
- Control system
  - $\pi_{0.5}$ directly commands target poses for the arms, gripper, and torso lift, and the target base velocities at *50 Hz* (with action chunking)
  - without any additional trajectory planning or collision detection
  - All manipulation and navigation control is fully end-to-end

$\pi_{0.5}$: 성능 평가

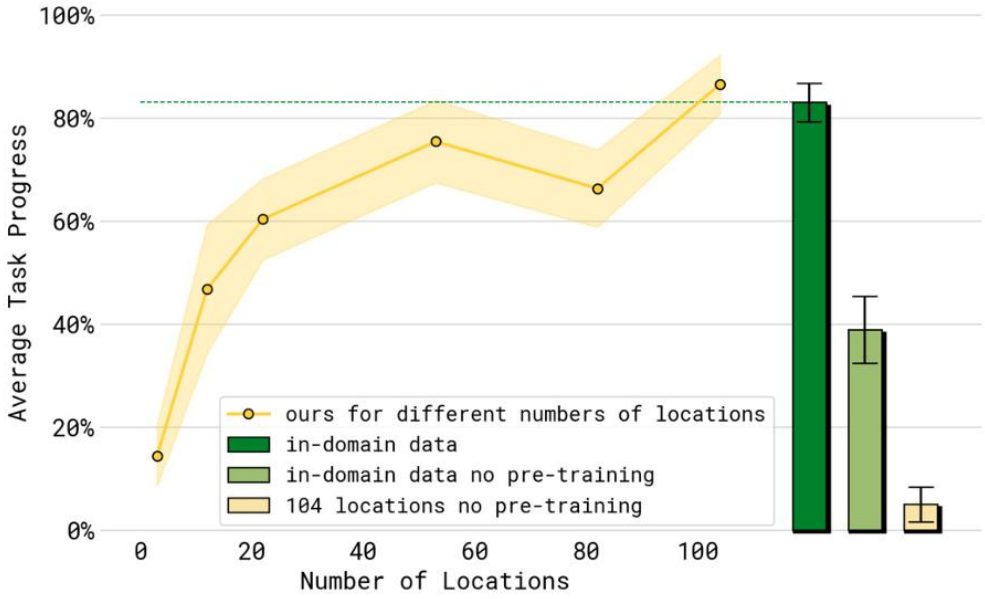
- Can $\pi_{0.5}$ effectively *generalize* to complex *multi-stage tasks* in entirely new homes?
- How does the generalization of $\pi_{0.5}$ scale with the number of distinct environments in the training data?
- How do the individual *co-training* ingredients in the $\pi_{0.5}$ training mixture contribute to its final performance?
- How does $\pi_{0.5}$ compare to the π_0 VLA?
- How important is the *high-level inference* component of $\pi_{0.5}$, and how does it compare to flat, low-level inference as well as oracle high-level baselines?

- Can $\pi_{0.5}$ generalize to real homes? Yes
  - able to consistently succeed on a variety of tasks in each home
  - Many of the tasks *involve multiple stages* (e.g., moving multiple objects) lasting about *2 to 5 minutes*
  - For these trials, the model is provided with a simple high-level command (e.g., “place the dishes in the sink”), and the *high-level inference* process autonomously determines appropriate steps (e.g., “pick up the cup”).

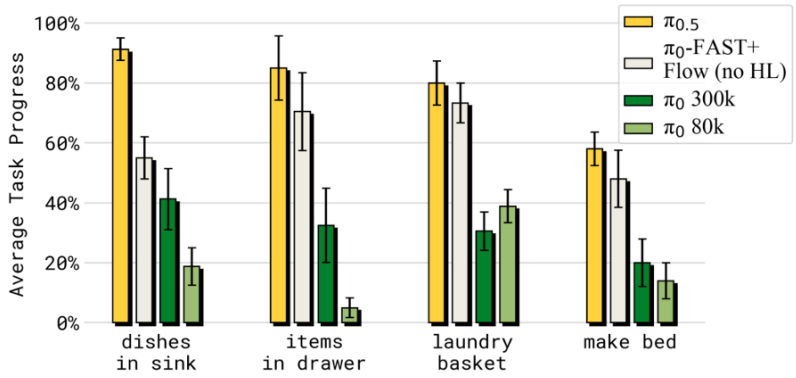




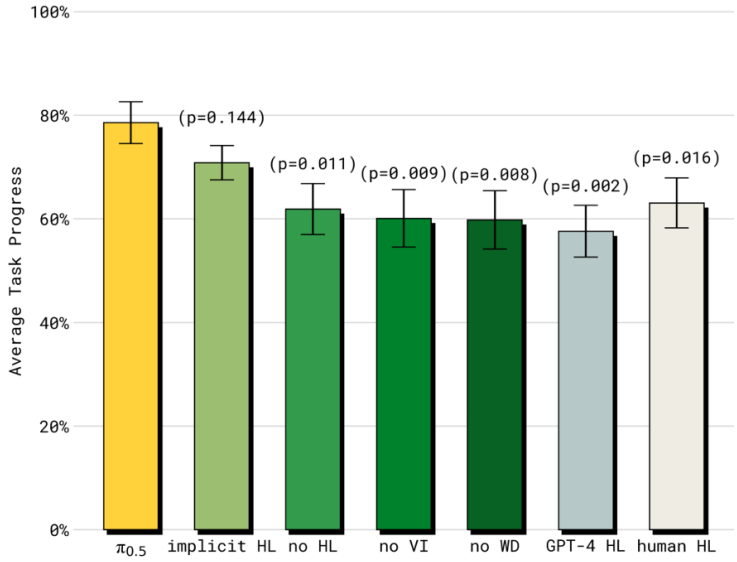
- How does generalization scale with the number of scenes?
  - Performance over the four test tasks — “dishes in sink”, “items in drawer”, “laundry basket”, “make bed” — *improves with* more training environments
  - Dashed green line and green bar show a baseline model that includes the test homes in the training set
  - Compared to this model, our best model achieves similar performance, despite not seeing any data from the test homes.



- How does $\pi_{0.5}$ compare to other VLAs?

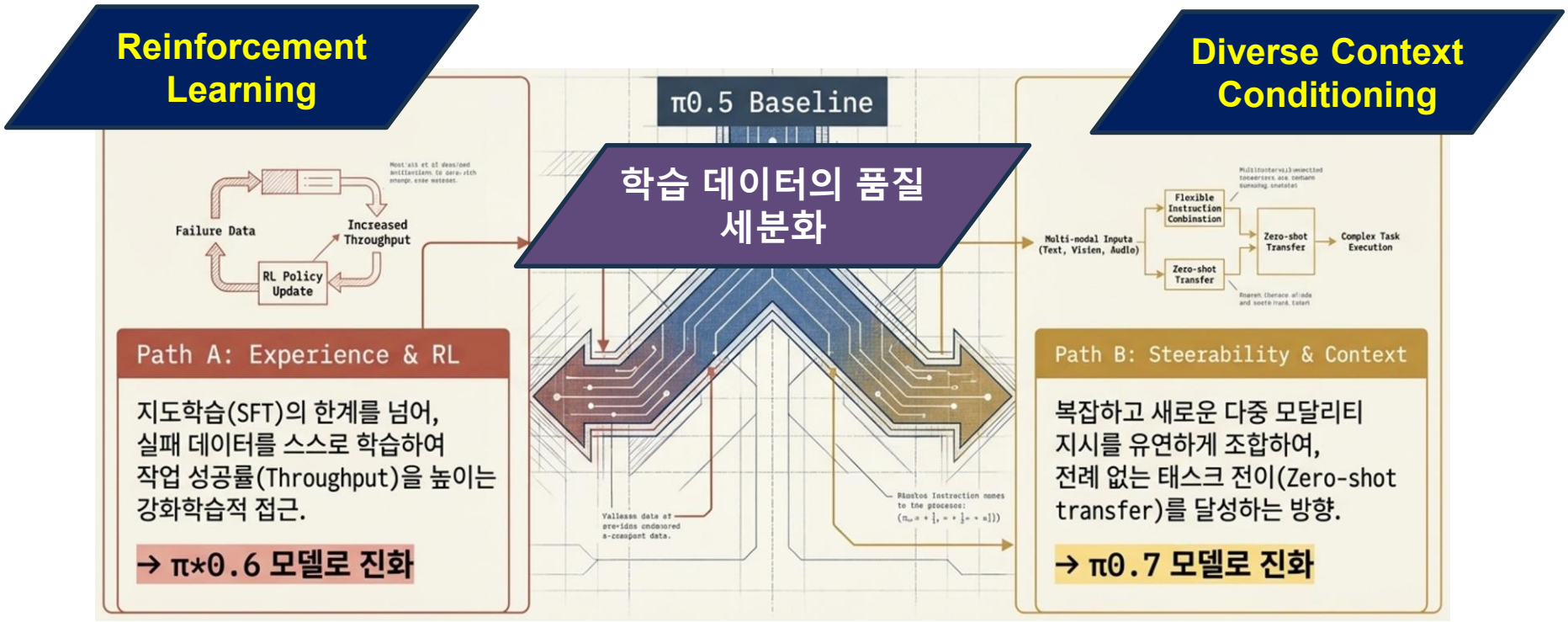


- How important is high-level inference?
  - no WD (Web Data)
  - No VI (Verbal Instruction)
  - Implicit HL
    - no high-level inference at runtime but includes high-level data in training, which may teach the model about subtasks implicitly
  - No HL (no high-level inference and data)
  - GPT-4 (for high-level policy)
  - Human HL
    - use an expert human as an “oracle” highlevel policy, to provide an upper bound on performance



From $\pi_{0.5}$: Evolution Strategies

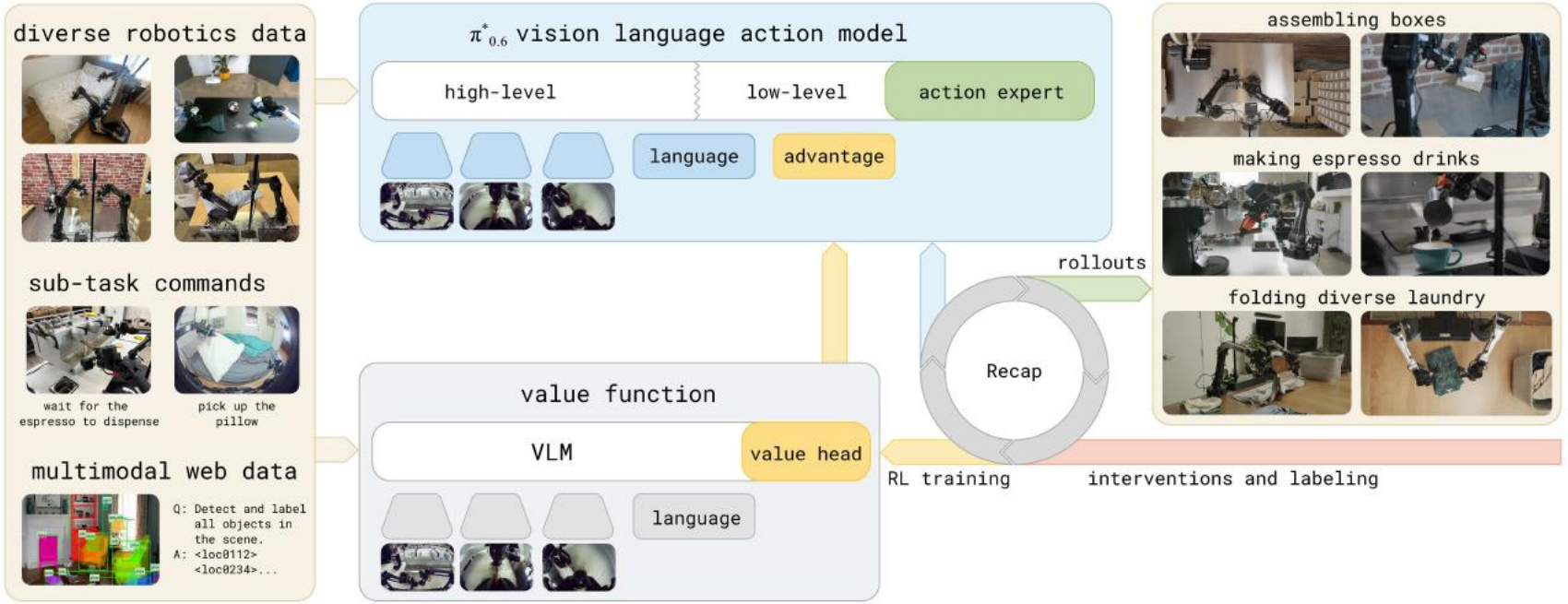
- $\pi_{0.5}$: Shows Possibility of Open-World Generalization
- For General Robot
  - More evolution is required



$\pi^*_{0.6}$: π + 강화학습 https://www.pi.website/blog/pistar06

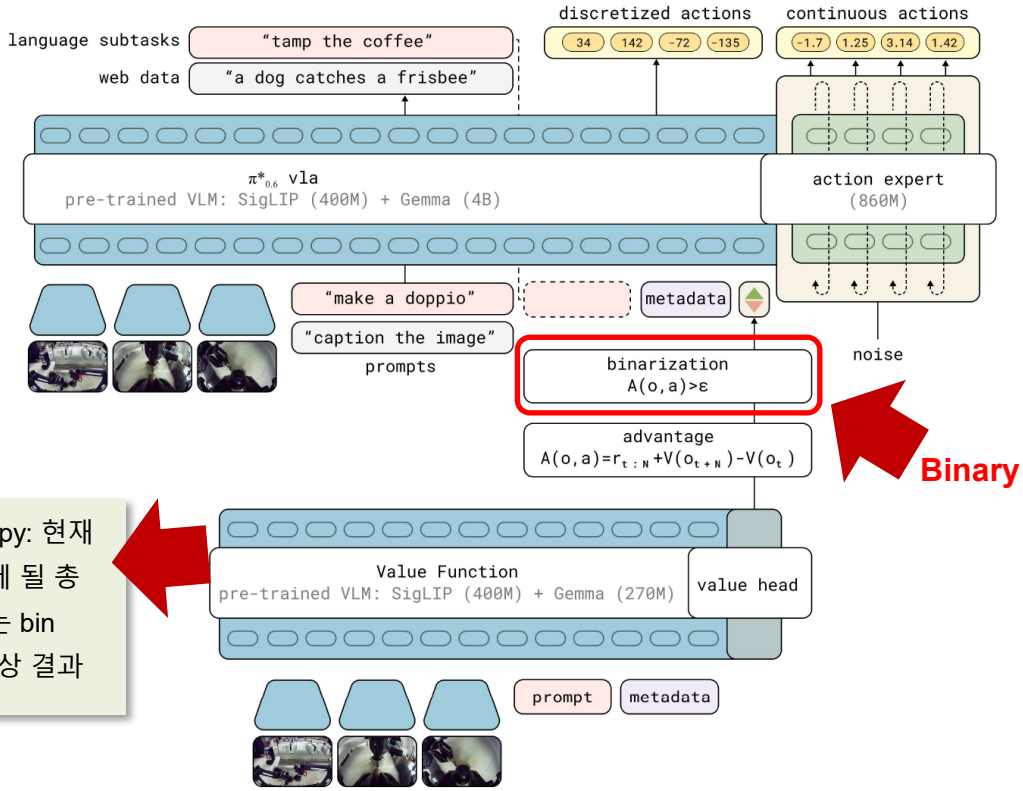
- π 계열의 논문들 중 강화학습을 VLA 학습에 사용한 첫 번째 연구
- Idea
  - 행동의 품질을 고려 (good/bad)하여 학습
    - 좋은 행동은 더 많이, 나쁜 행동은 더 적게 선택하도록 (or 오답노트로 활용하도록) 학습
  - 좋지 않은 episode data도 활용
- 학습 방법
  - Pre-Training 단계
    - 수많은 로봇 데이터와 웹의 시각-언어 데이터를 통해 general한 능력을 갖춘 VLA가 되도록 학습
  - SFT (Supervised Fine-Tuning) 단계
    - 모델이 특정한 task를 잘 수행할 수 있도록 expert dataset으로 fine-tuning
  - Post-Training & RL
    - Fine-tunin된 모델을 바탕으로 스스로 환경에서 해당 작업을 반복적으로 수행하며 데이터(autonomous dataset)를 수집
    - 잘못된 동작을 할 경우 사람이 개입하여 이를 바로잡아주는 교정(correction) 데이터가 추가
    - 수집된 데이터를 바탕으로 가치 함수(value function)를 업데이트하고, Advantage Label (positive/negative)을 계산하여 정책을 정교화
    - 이 과정을 반복

$\pi^*_{0.6}$: RECAP Framework (경험 학습)



- Overcome SFT (Supervised Fine Tunning)
  - SFT: 실패 상황에서 스스로 회복하는 능력 부족
- RECAP (RL with Experience and Corrections via Advantage-conditioned Policies) Feedback Cycle
  - Robot의 자율적 경험 (Rollout)을 수집하여 throughput 향상
    - Data 수집에 사람의 개입 최소화

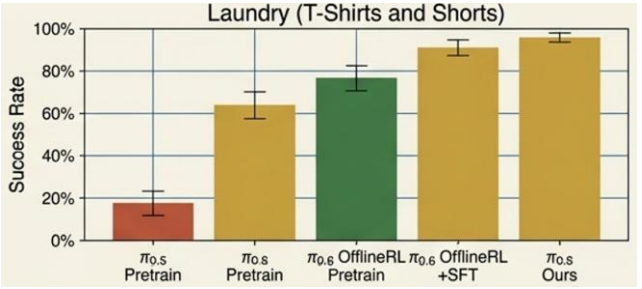
$\pi^*_{0.6}$: Advantage Conditioning



Binary Conditioning

- Training Loss → Cross Entropy: 현재 상태의 궤적을 따라 달성하게 될 총 보상이 속할 것으로 예상되는 bin (201개)과 ground truth의 보상 결과

- Value Function: 상태와 품질 평가를 위한 별도의 VLM 학습
  - **Value = task 성공까지 남은 step수**
  - **Sparse Reward: 성공(0), 남은 step 마다 -1, 실패시 큰 감점**
- Binary Conditioning
  - Rollout 품질을 Advantage (True/False)로 치환하여 모델 input context로 제공
- Reinforce Actions
  - 현재 상태에서 Advantage가 True일 때의 패턴을 선호하도록 (행동을) 유도
  - SFT 대비 실패율 최대 2배 감소



$\pi_{0.7}$: Steerable Robot Foundation Model

https://www.pi.website/blog/pi07

- 목적: Compositional generalization을 통한 zero-shot 성능 향상
  - enabling it to follow diverse language instructions,
  - attain performance comparable to more specialized fine-tuned models on dexterous tasks,
  - even compose these behaviors in new ways
- 방법
  - leveraging *large and diverse datasets*, including
    - data from many robots with *diverse* strategies,
    - suboptimal data from autonomous execution (including both data from RL post-trained agents as well as *failures*), and
    - *non-robot data* from videos of humans performing tasks and general multimodal data from the internet
  - *annotating the data* with detailed ***context*** annotations that
    - contain not only information about *what* to do but also *how* to do it and *how good* was an episode
    - provide this knowledge to the model using a variety of *multimodal conditioning signals*
  - Prompt (or Context) Structure
    - detailed language labels, strategy metadata, and multimodal information such as subgoal images
  - 의도
    - teaches the robot about *nuanced concepts and skills* that it could use not only to perform the training tasks effectively but also to *compose in new ways* to solve new tasks.

$\pi_{0.7}$: Overview

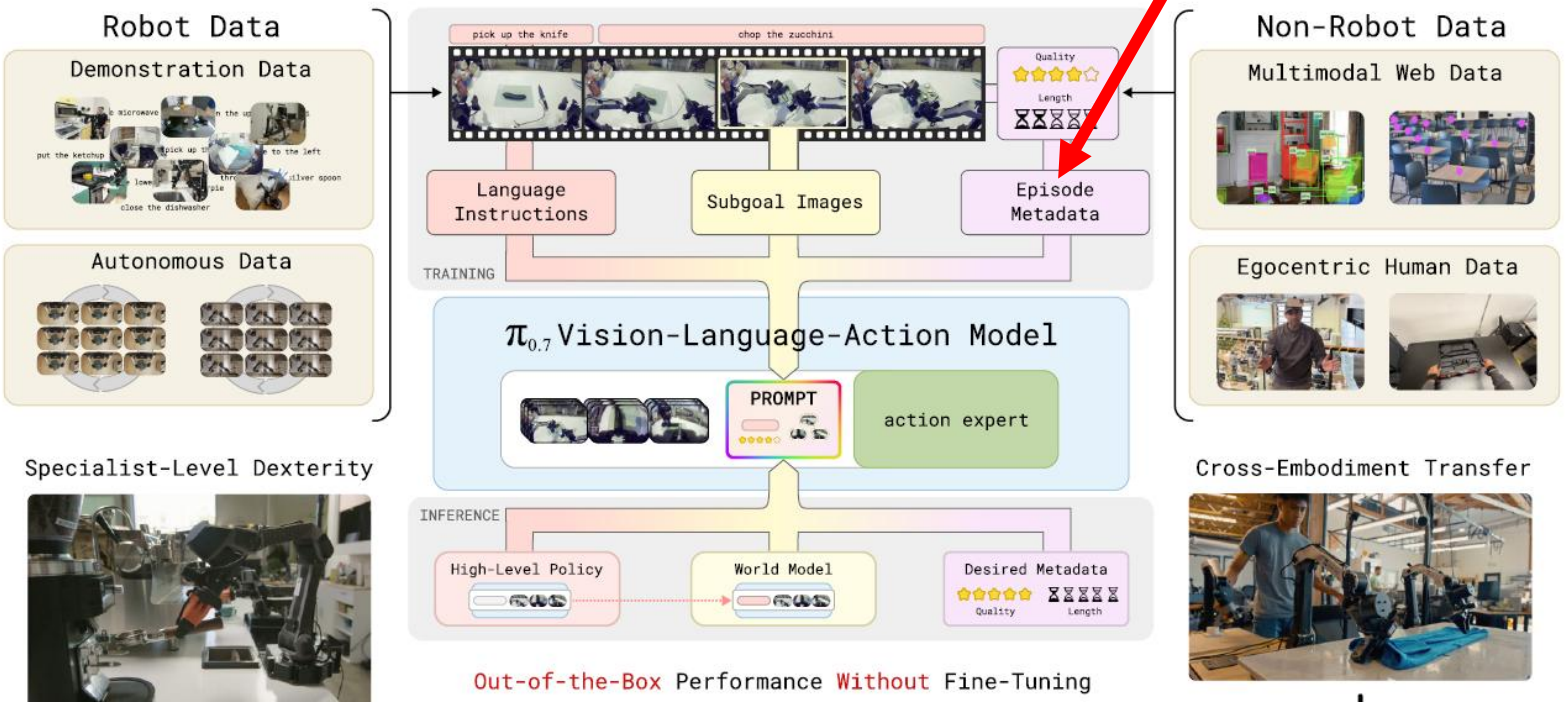
- trained with diverse prompts that contain not only the task description, but detailed language, generated subgoal images, and episode metadata

$\pi_{0.7}$ shows

- Out of the box performance: without any task specific post-training and in a variety of environment.
- Instruction generalization: robust generalization to complex, unseen language references.
- Cross-embodiment generalization
- Compositional task generalization: can be instructed to perform new tasks by composing skills in previously unseen ways

Episode metadata (data quality and strategy)

- Speed: 작업에 걸린 episode 길이
- Quality: 1~5점
- Mistake: 해당 행동 구간에서 실수 발생 여부
- Control Mode: joint or EE 제어방식



$\pi_{0.7}$: Examples

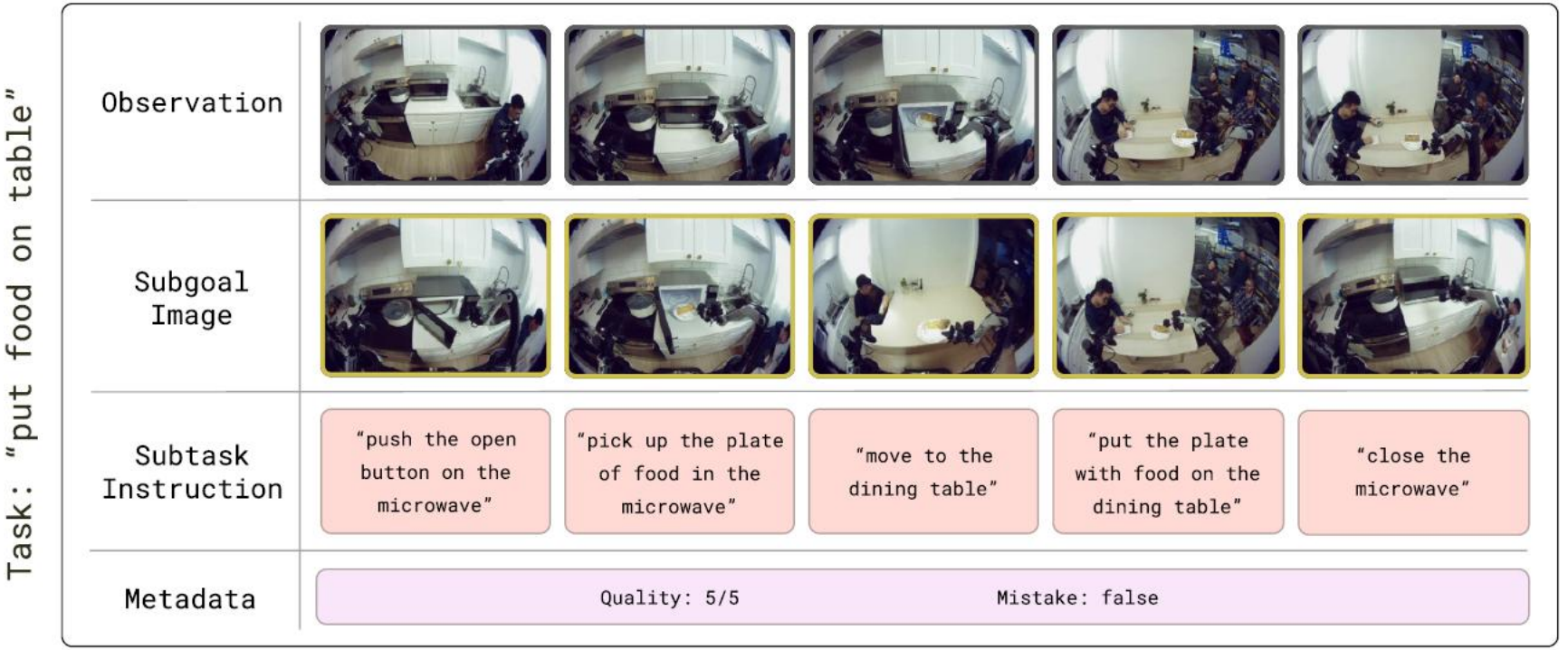
- Example Language memory (long-horizon) update

m_t: I placed a plate in the cabinet and moved to the counter.

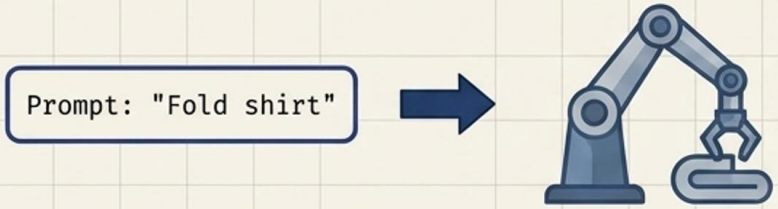
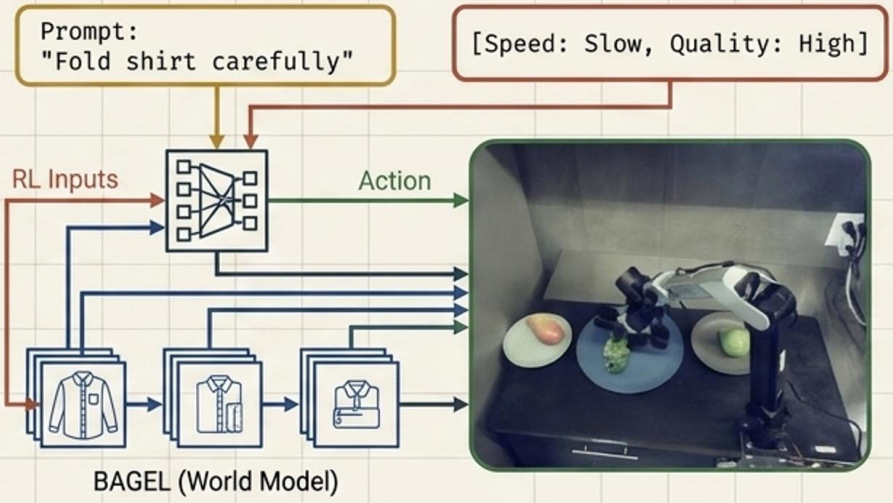
$\Downarrow$

m_{t+1}: I placed a plate in the cabinet, moved to the counter, and picked up a bowl.

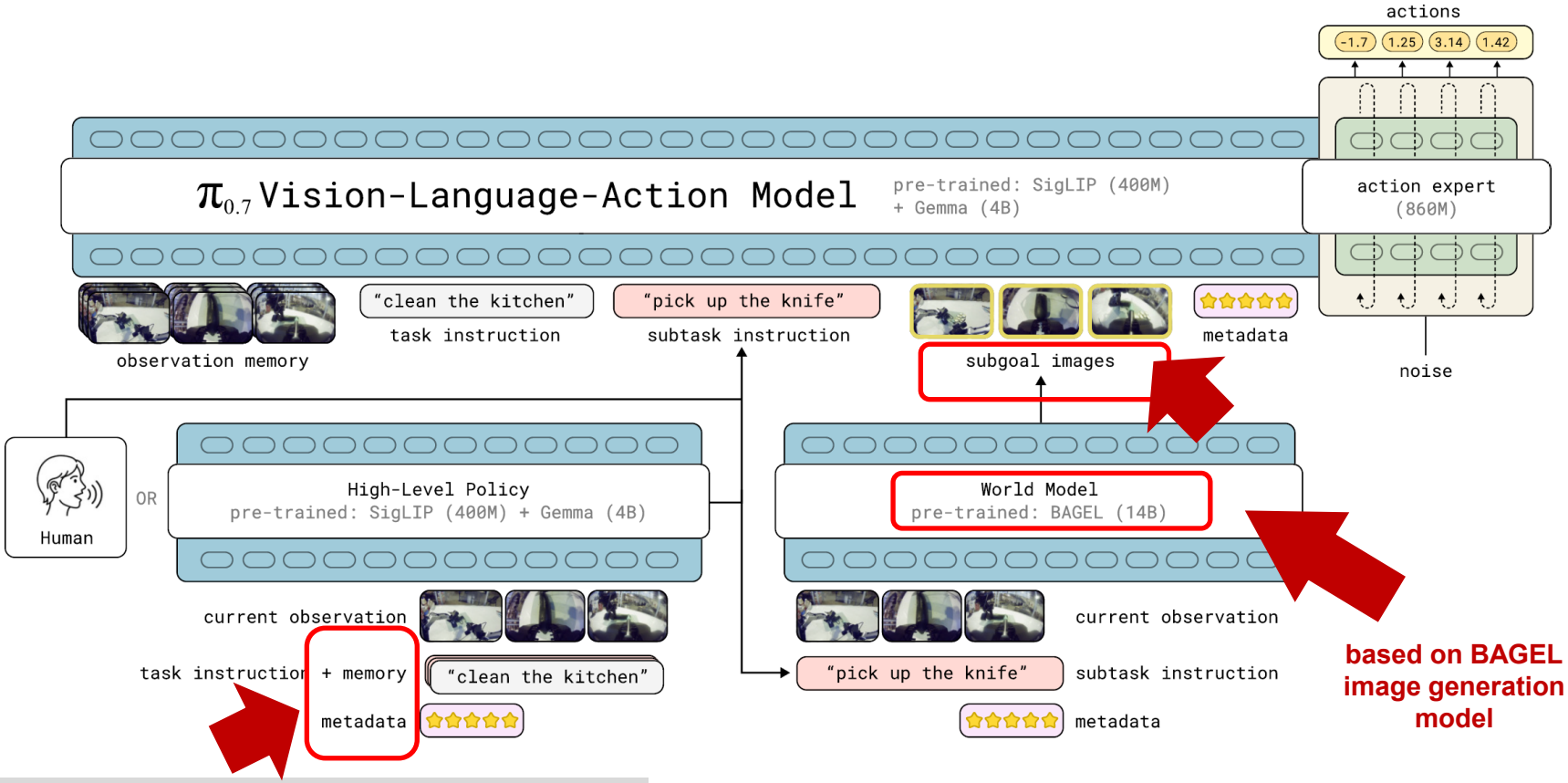
- Prompt Example



Comparison

Standard VLA	π0.7 Architecture
	
단일 속도 및 품질로만 실행 가능.	Episode Metadata를 통한 전략적 행동 조절 및 BAGEL(이미지 생성 모델) 기반 Subgoal 이미지를 통한 전례 없는 작업 조합(Composition) 지시.

$\pi_{0.7}$: Architecture Overview



- $a_{t:t+H}$: continuous robot actions from t to t+H
- $o_{t-T:t}$: observation image from t-T to t
- g: task goal
- π_{LL}, π_{HL} : low-level and high-level policies
- $o_{t-T:t}$: ***image short-horizon memory*** (K<<T)
- l_{t+1}: subtask instruction
- m_t: previous semantic events in natural language (***language (long-horizon) memory***)

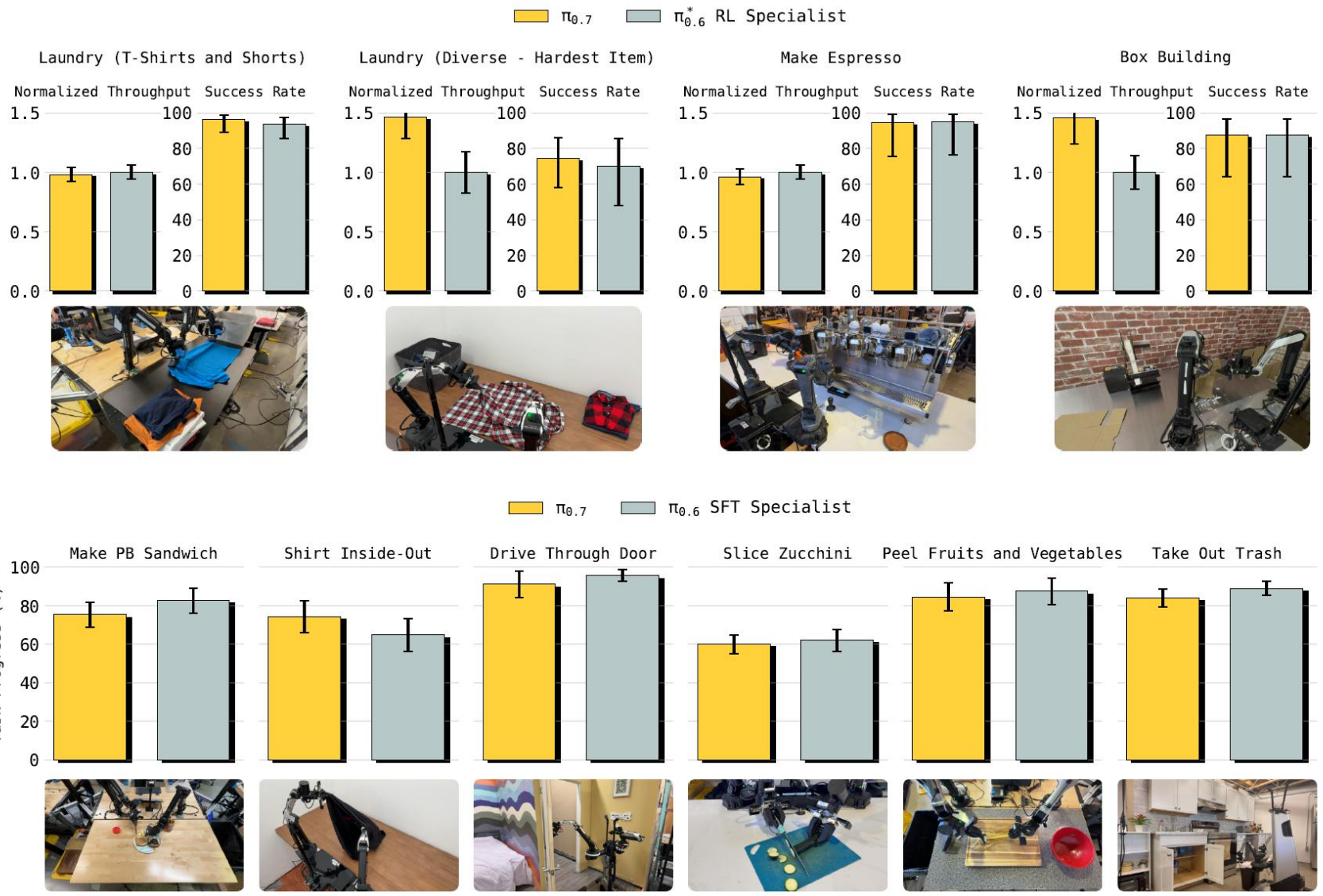
Goal: $\pi(a_{t:t+H}|o_{t-T:t}, g)$

Factorization:

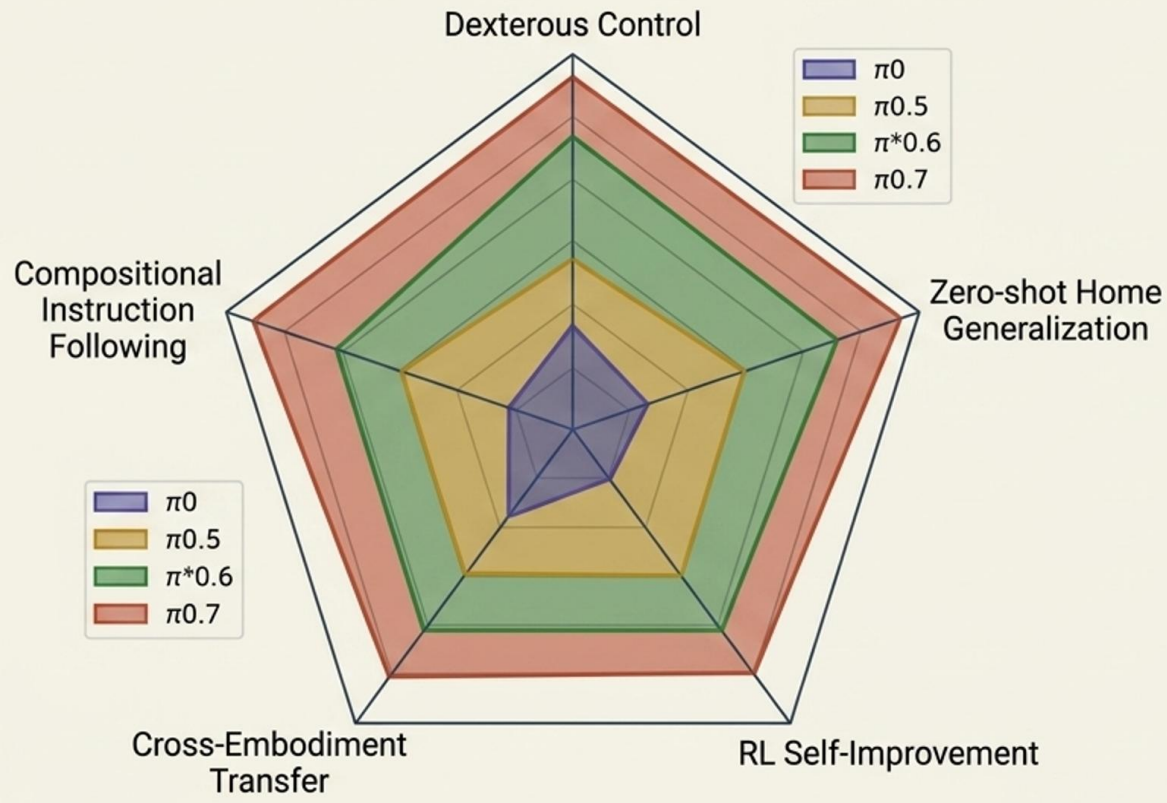
$$\pi(a_{t:t+H}, l_{t+1}, m_{t+1}|o_{t-T:t}, m_t, g)$$
$$\approx \pi_{LL}(a_{t:t+H}|o_{t-K:t}, l_{t+1}, g)\pi_{HL}(l_{t+1}, m_{t+1}|o_t, m_t, g)$$

(ref) MEM: Multi-Scale Embodied Memory for Vision Language Action Models (https://arxiv.org/pdf/2603.03596)
BAGEL → Emerging Properties in Unified Multimodal Pretraining (https://arxiv.org/abs/2505.14683)

$\pi_{0.7}$: Out-of-the-Box Dexterity



	π_0	$\pi_{0.5}$	$\pi^*_{0.6}$	$\pi_{0.7}$
VLM Backbone	PaliGemma (3B)	SigLIP+Gemma (2.6B)	Gemma 3 (4B)	Gemma 3 (4B) + MEM History
Action Expert Size	300M (Flow Matching)	300M (Hybrid)	860M	860M
Key Innovation	정밀 제어 (50Hz)	계층적 추론	가치 함수 기반 RL	다중 모달리티 프롬프팅
Key Context	Text Only	Sub-task Prediction	Binarized Advantage	Metadata + Subgoal Images

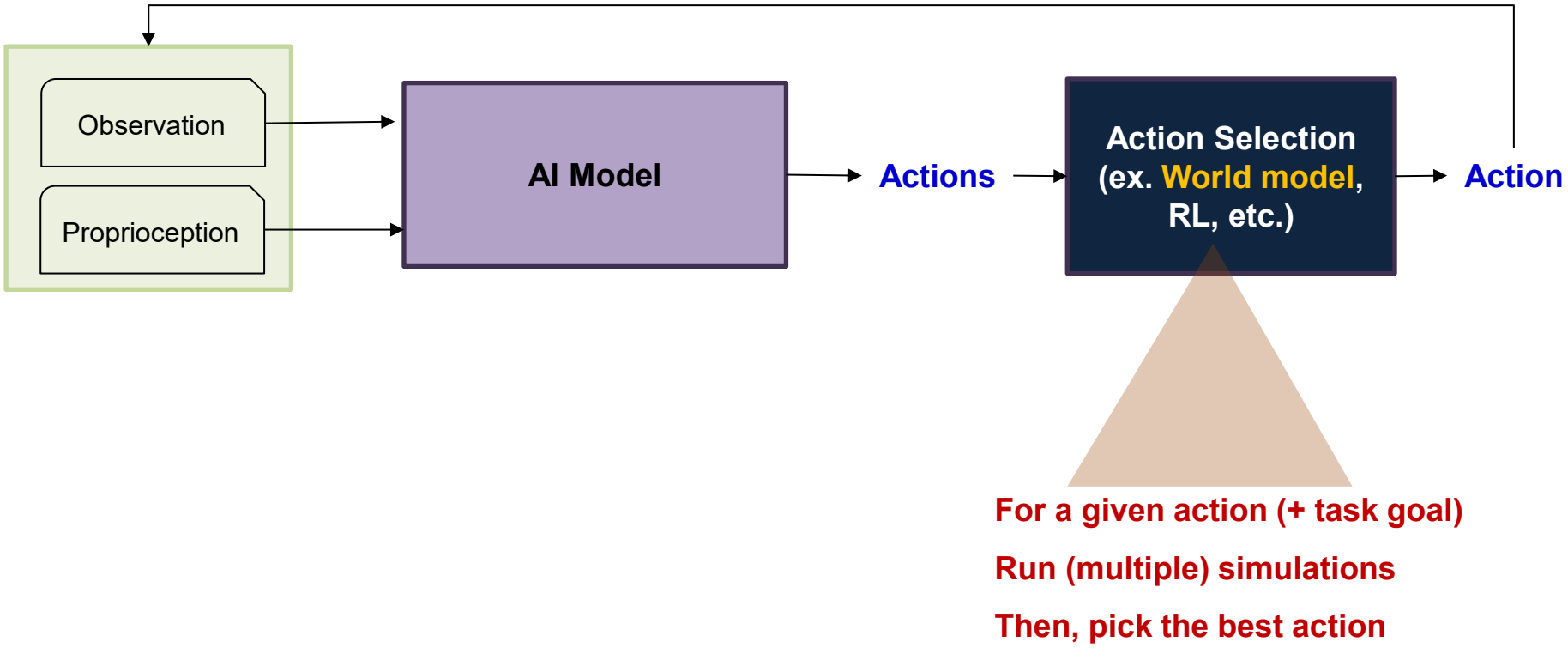


- **π0**: 기본 Dexterity와 Cross-Embodiment의 토대 마련.
- **π0.5**: Open-world (Zero-shot Home) 환경 성능에서의 급격한 팽창.
- **π*0.6**: RL을 통한 최적화 스펙트럼에서 압도적 우위 확보.
- **π0.7**: 복잡한 다중 지시와 조합(Composition)에서 최종적인 확장 달성.

World (Foundation) Model

- A world model in AI is a machine learning system that
  - builds an internal representation of the physical environment,
  - learns its physics, spatial dynamics. Ex)
    - 중력의 영향으로 물체는 아래로 떨어진다는 사실
    - 시야에서 잠시 사라져도 객체는 계속 그 자리에 존재한다는 항상성 (Object Permanence)
    - 한번 움직인 물체는 계속 나아가려는 관성
    - 물이나 연기가 자연스럽게 확산하는 유체 역학의 기초적인 원리
  - learns causality. Ex)
    - '공을 차면 날아간다' 또는 '스위치를 누르면 불이 켜진다'와 같이 명확한 행동과 결과로 이어지는 인과 관계 (causal relation)
  - Instead of just matching patterns, it *simulates how the environment changes and predicts future states* to make decisions

- World Model Examples
  - Google Genie
    - https://www.youtube.com/watch?v=PDKhUknuQDg&t=31s
  - Nvidia Cosmos
    - https://www.youtube.com/watch?v=9Uch931cDx8&t=2s
  - Meta V-JEPA
    - https://ai.meta.com/research/vjepa/


Observation
Proprioception
AI Model
Actions
Action Selection
(ex. World model,
RL, etc.)
Action
For a given action (+ task goal)
Run (multiple) simulations
Then, pick the best action